

Building and Consuming a Database-Bound RESTful Web Service

Contents

Part1: Building the RESTful Services	2
Step 1: Get the Application Ready	2
Step 2: Create the Department REST Service.....	9
Step 3: Create the Employee REST Service	18
Part 2: Developing the Client Application	26
Step 1: Create the Department REST Data Controls	26
Step 2: Create the Employee REST Data Controls	33
Step 3: Build a Master-Detail Page	44
Step 4: Run and Test the Application	51
Part 3: Running the Complete Application.....	55
Step 1: Open and Run the Completed Client Application	55
Step 2: Test the Application	58

In following tutorial, you will build a master-detail Oracle ADF web application that queries data from the database through Enterprise JavaBeans (EJB)–backed REST services. To limit the scope of the Oracle JDeveloper 12c application you will build, some parts of the hands-on application have been prebuilt for you.

To access the departments and employees data, you’ll use two classes—DepartmentService and EmployeeService—to implement the REST web services. For the purposes of this tutorial, the code within each class will be abbreviated. In a real-world implementation, you would typically support the full CRUD methods for each resource.

This tutorial requires to have JDeveloper 12.1.3. installed and to have access to an Oracle database 10g and onward with access to the HR schema.

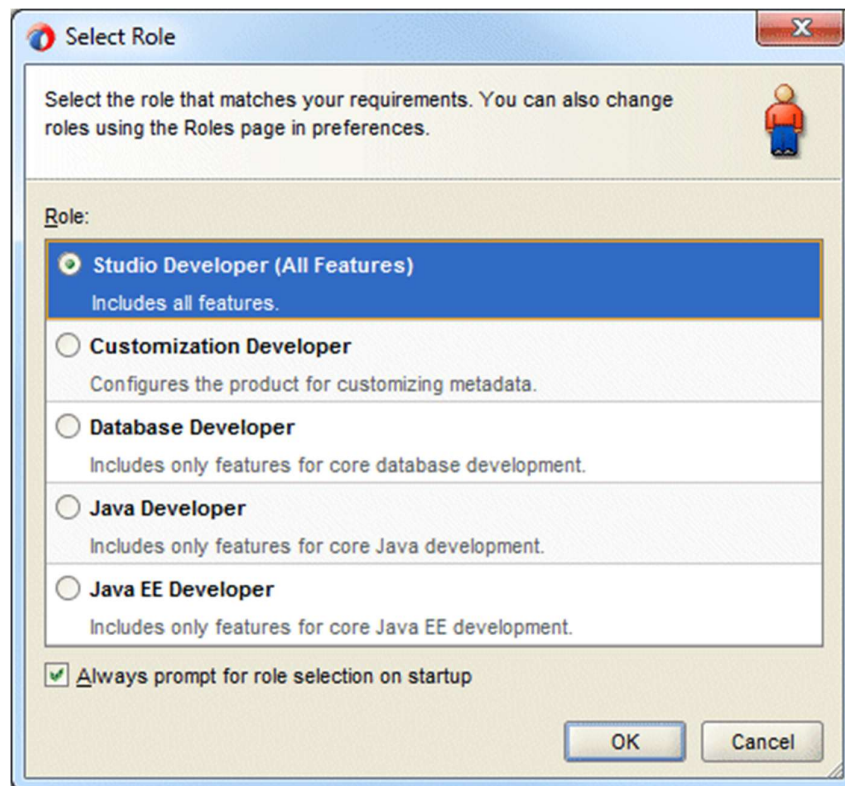
Part1: Building the RESTful Services

In this first section of the tutorial, you create two REST services from existing application POJOs.

Step 1: Get the Application Ready

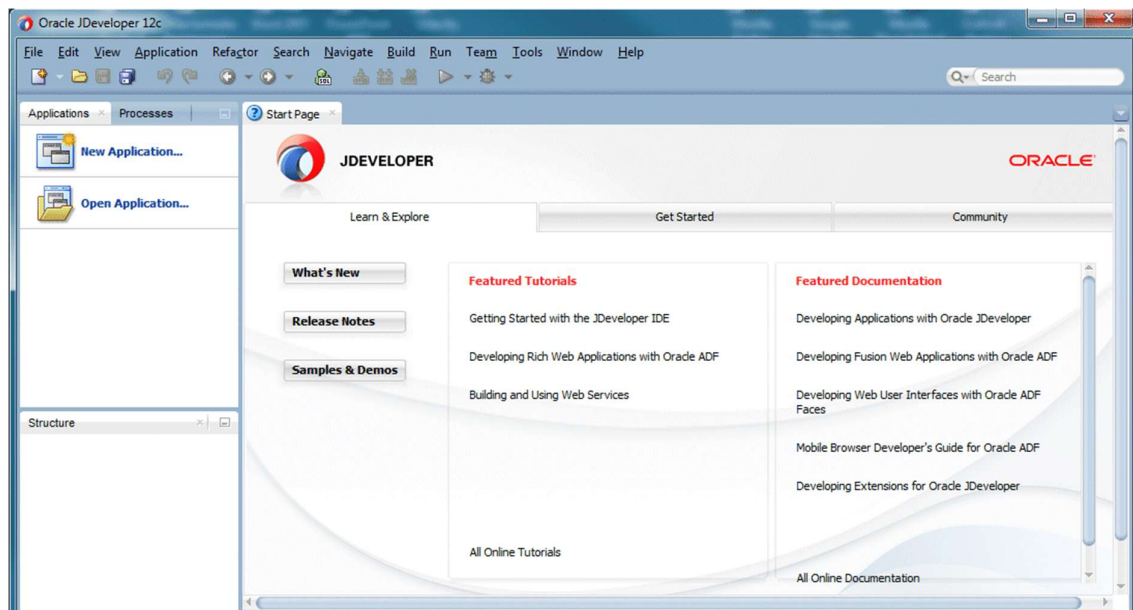
Before you create any components, you must first open the starter application and projects.

1. Start JDeveloper by selecting **Start > All Programs > Oracle Fusion Middleware 12.1.3.0.0 > JDeveloper Studio 12.1.3.0.0**
2. If prompted for a Role, select **Studio Developer (All Features)** and click **OK**.



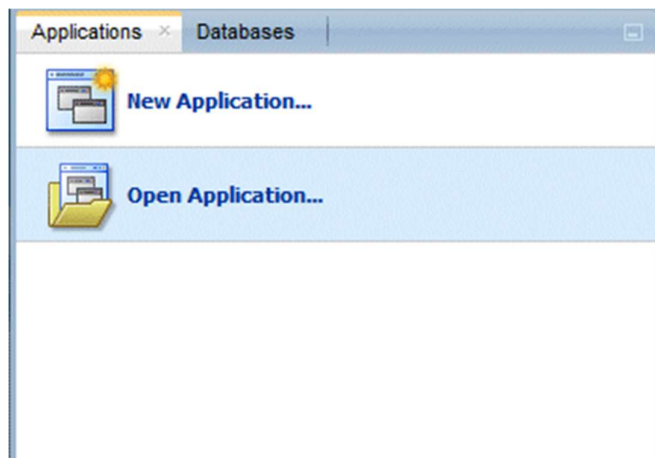
3. Close the **Tip of the Day** window.

Once loaded, the JDeveloper IDE appears. The very first time you open JDeveloper, the Start Page displays. You can re-invoke the Start Page later by choosing **Help > Start Page**.

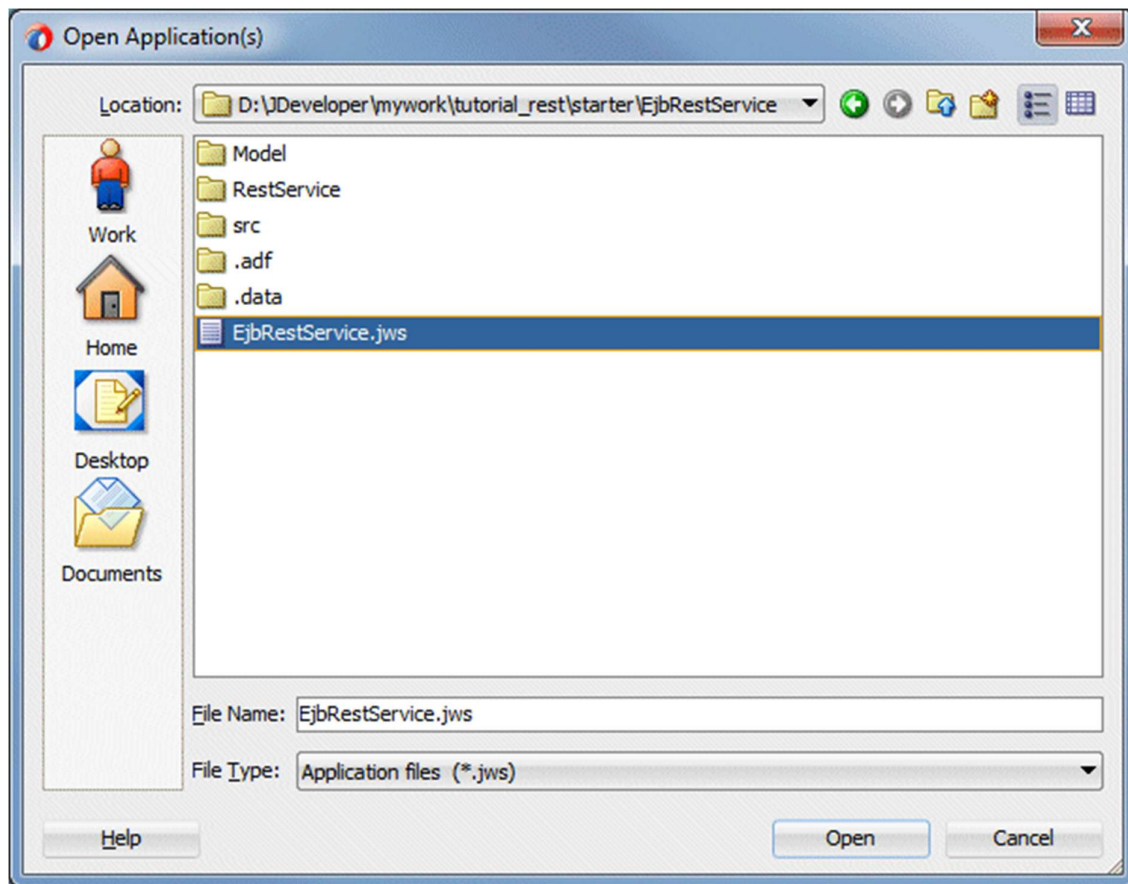


Notice the various options available to help you learn about JDeveloper. After exploring these options, click the **X** on the Start Page tab to close it. (the X appears when you mouse over the tab).

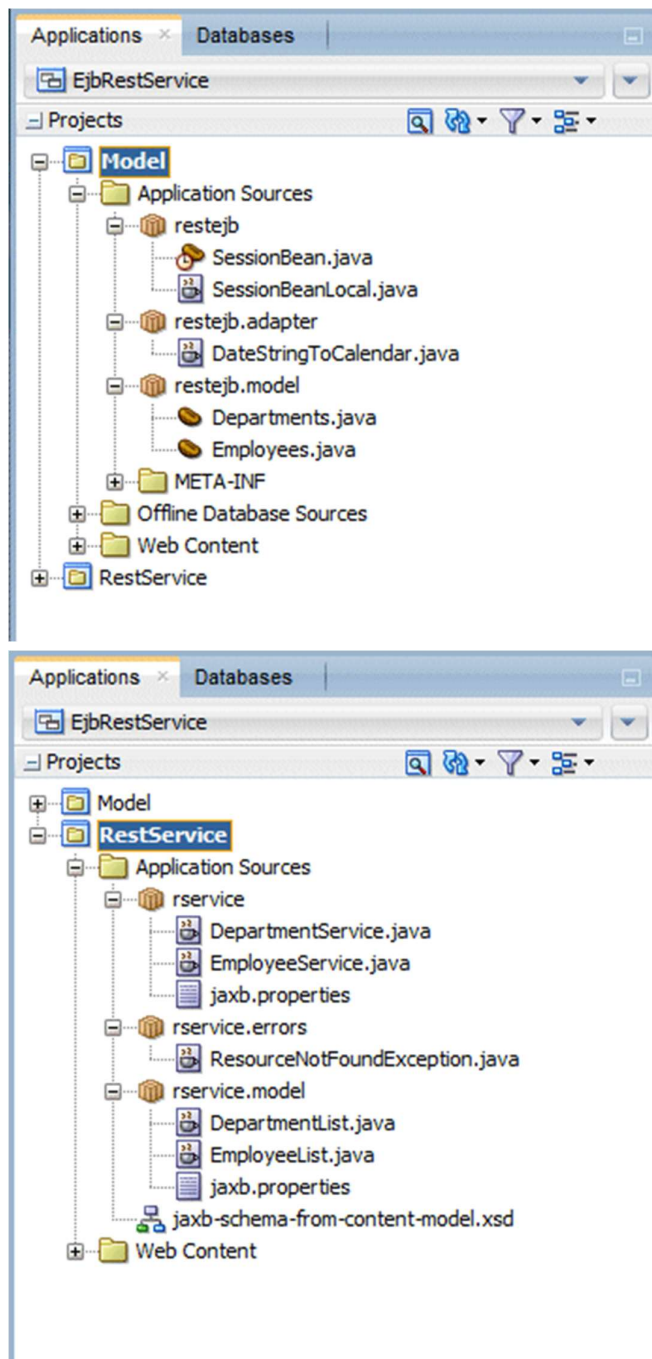
4. Click the **Application** tab to go back to the Applications window. Click the **Open Application** icon.



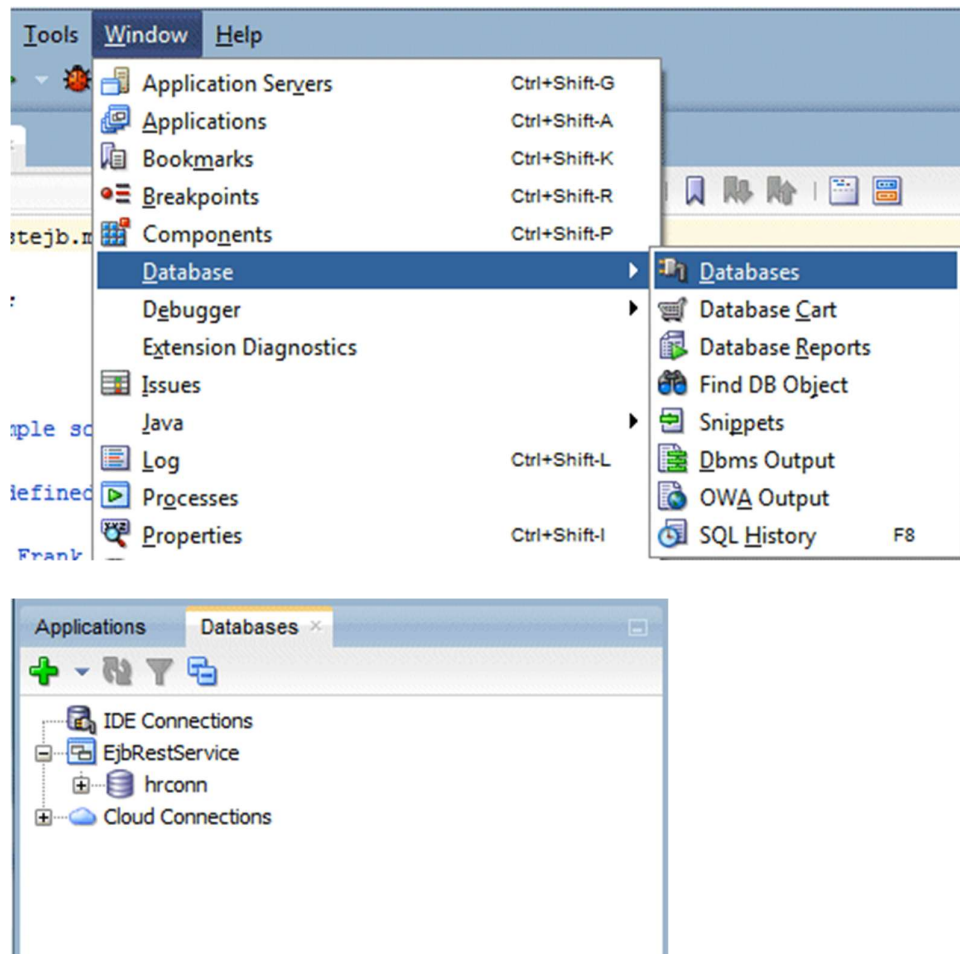
5. In the Open Application(s) window, navigate to the location where you unzipped the starter application and select the `...\EjbRestService\EjbRestService.jws` file. Then click **Open**.



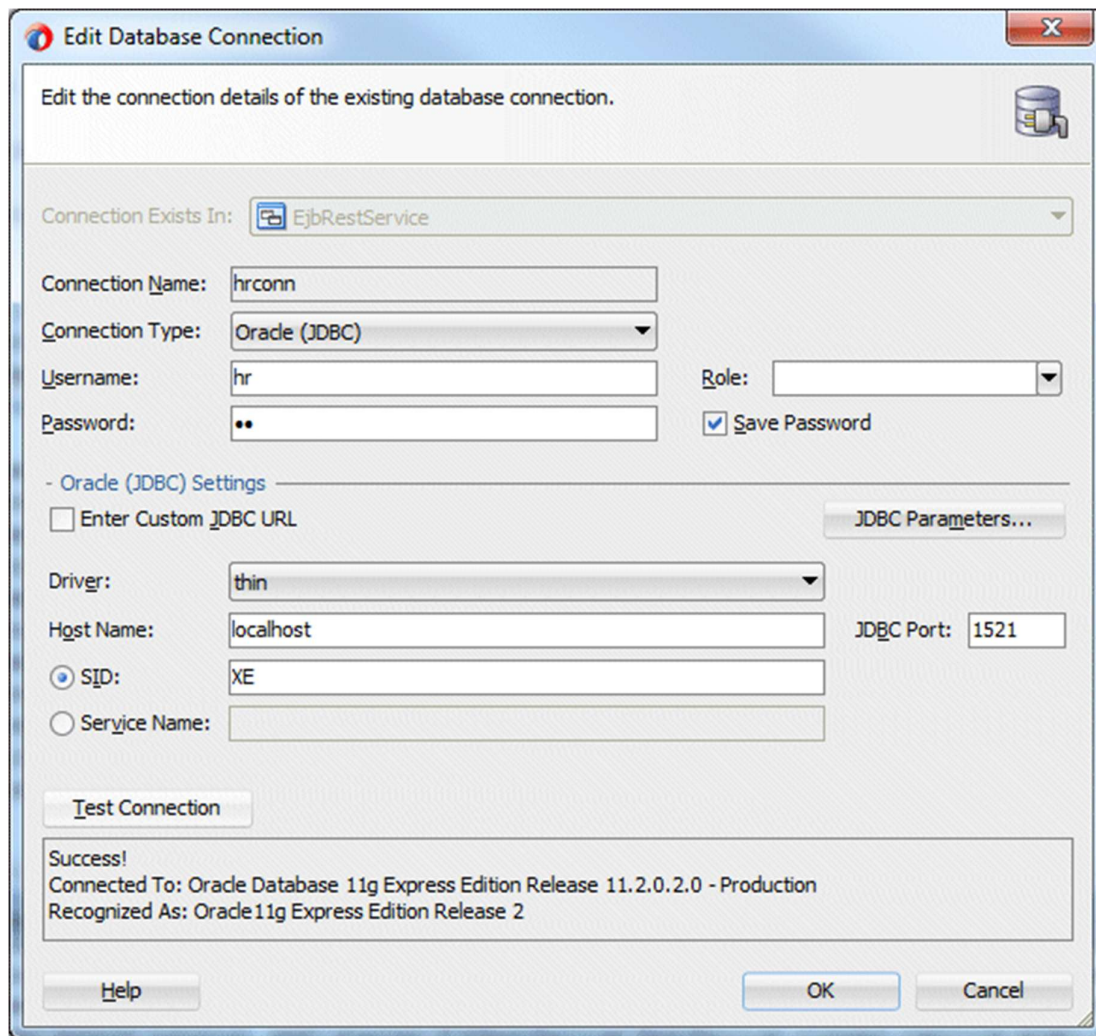
6. In the Application window, you should see two projects **Model** and **RestServices**.
The Model project contains the data classes and session bean.
The RestServices project contains the services classes and helper methods.



7. Next, set the database connection to point to you HR schema. Select **Window > Database > Databases**, and expand the **EjbRestService** node to display the **hrconn** node.

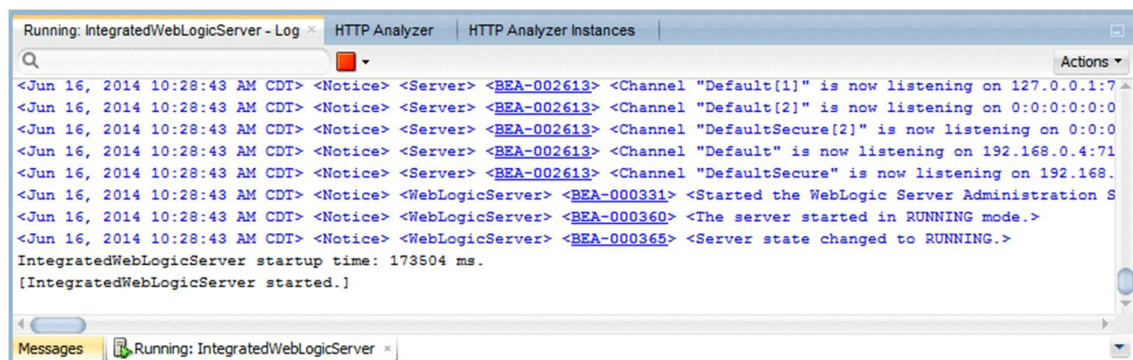
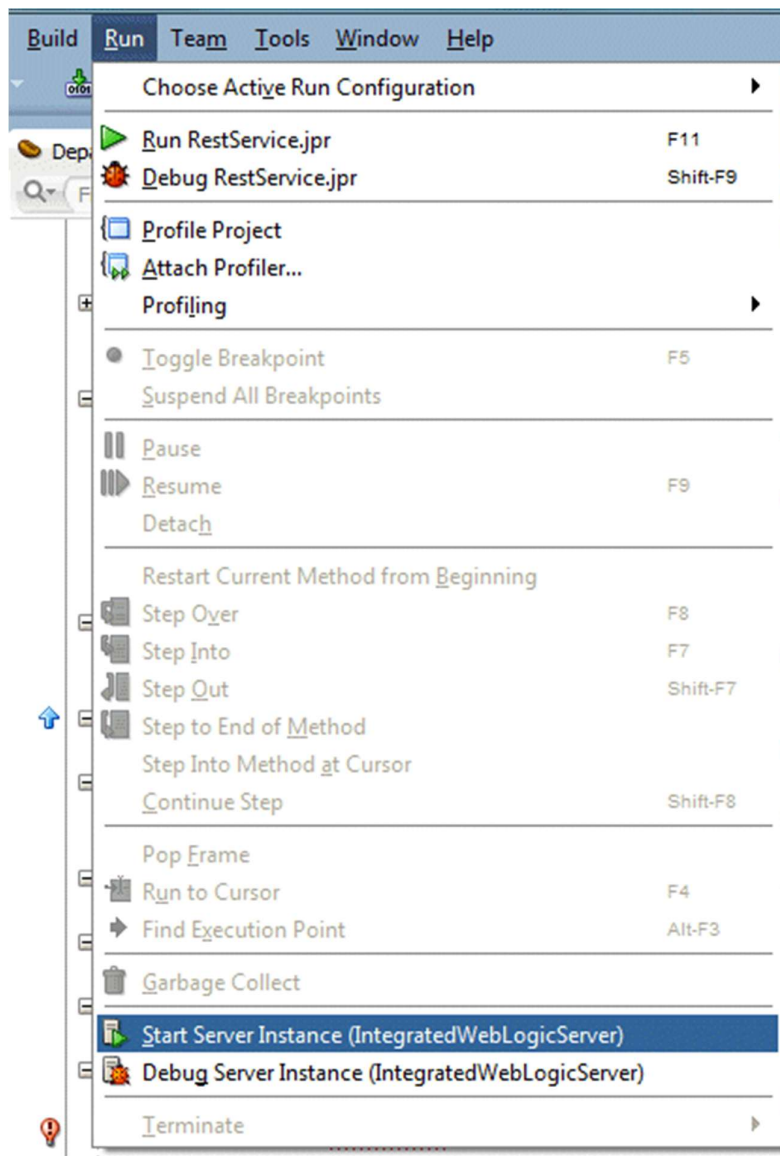


- Right-click **hrconn**, and select **Properties** from the menu. Edit the database connection information to work with your database setup. Test the changes (click **Test**), and click **OK**.



- Next, start the Oracle WebLogic Server instance integrated with Oracle JDeveloper. To start the Oracle WebLogic Server, select **Run > Start Server Instance**. If this is the first time you've run the integrated Oracle WebLogic Server, a Create Default Domain dialog box will open. Create a password for the default Oracle WebLogic Server domain.

Note: To ensure that the integrated Oracle WebLogic Server listens for localhost and the IP address of your computer, don't select an address from Listen Address..



10. Wait for the integrated Oracle WebLogic Server to start before proceeding to the steps in the next section. The first time you start the server, it builds the Oracle WebLogic Server domain, which may take several minutes.

Step 2: Create the Department REST Service

In this section of the tutorial, you'll use Oracle JDeveloper's wizard for creating RESTful services to REST-enable the DepartmentService class and to expose the public `getAll()` method for GET access. The REST-specific annotations added to the Java file are from Jersey, the Java EE 6 JAX-RS REST Reference Implementation bundled with Oracle JDeveloper. Finally, you'll test the REST service, using Oracle JDeveloper's integrated HTTP Analyzer.

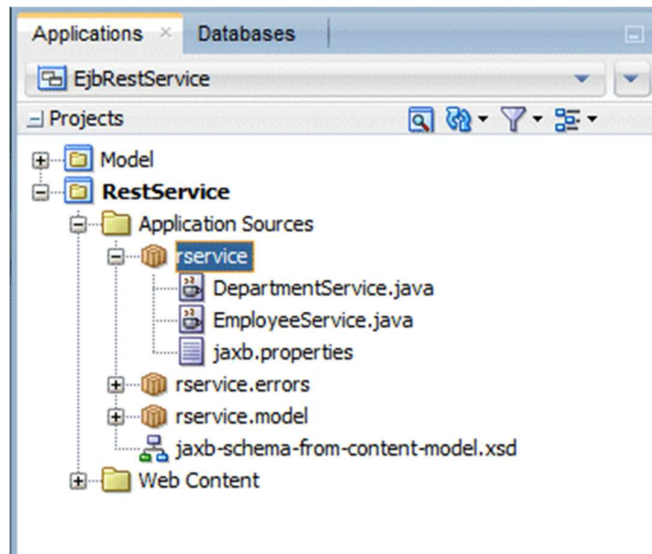
The EjbRestService workspace contains two projects:

The Model project uses Oracle TopLink and an EJB session bean to query and update the Departments and Employees tables in the HR schema.

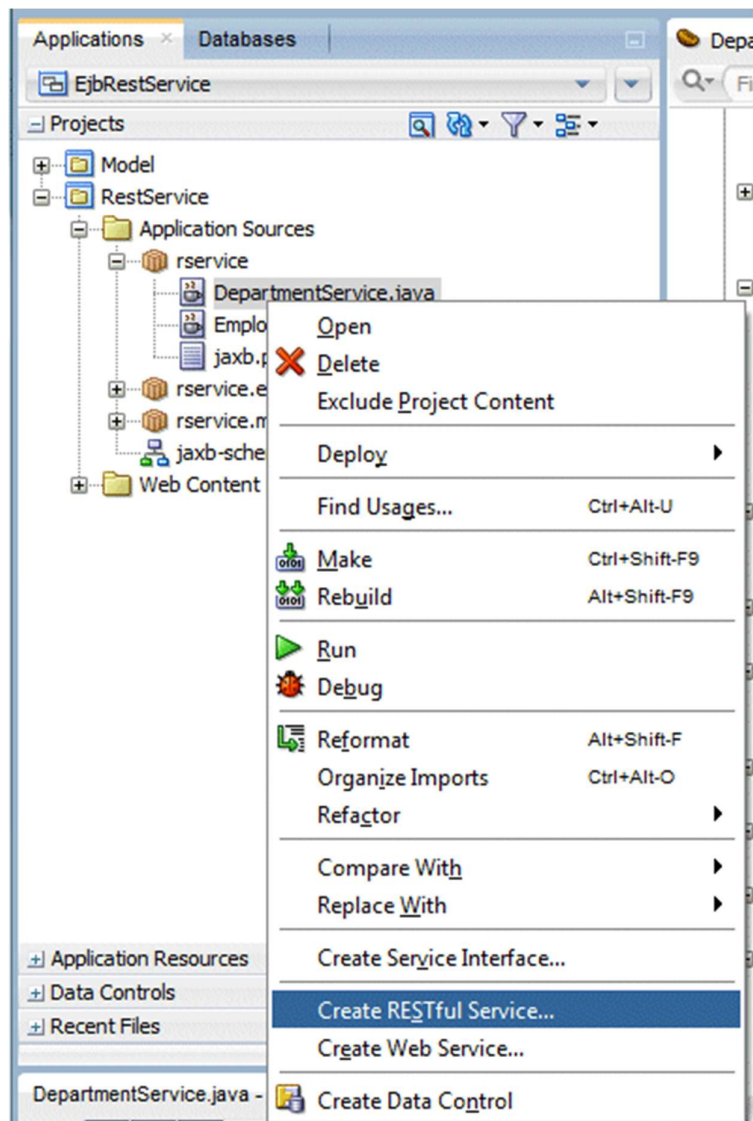
The RestService project references the EJB session bean in the Model project from two JavaBeans—`DepartmentService.java` and `EmployeeService.java`—to read and write data. The two JavaBeans represent the resources you will now expose as REST services.

To create the department service, do the following:

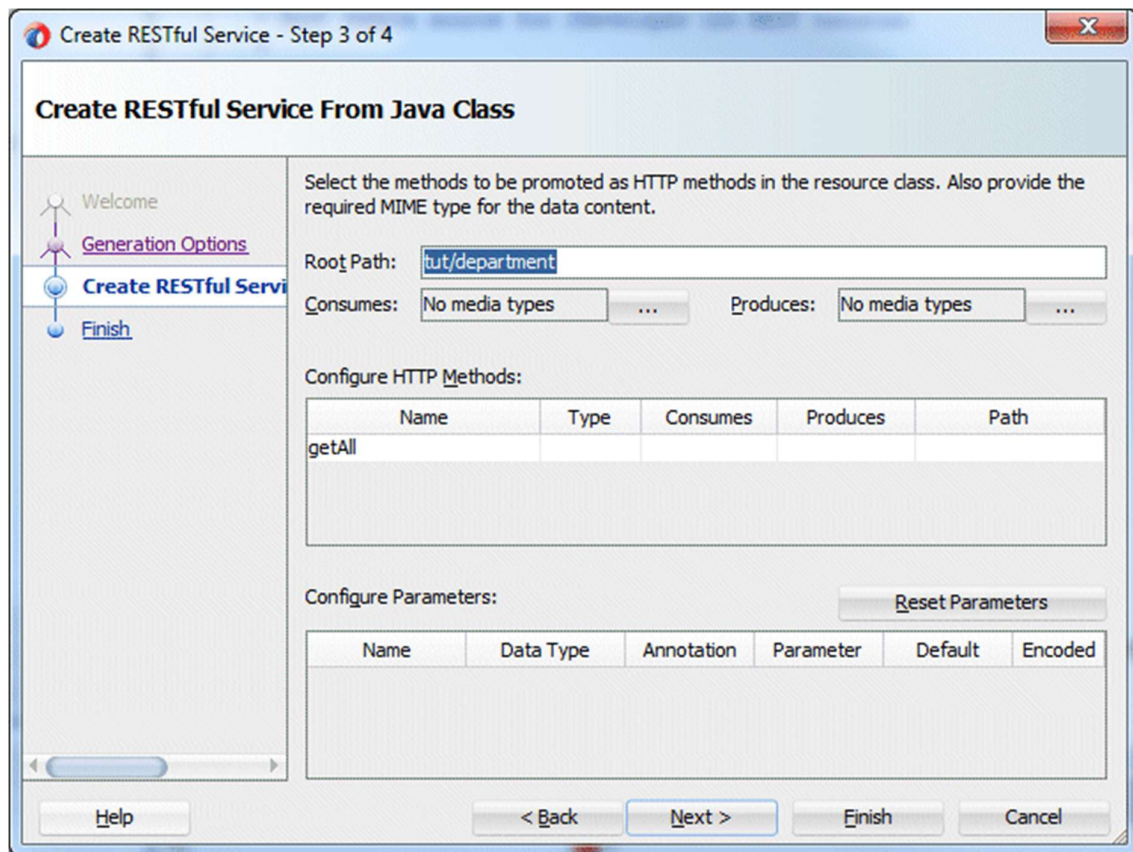
1. In the **Applications** window, expand the **RestService** project node, **Application Sources**, **rservice** package.



2. Right-click the **DepartmentService.java** file entry, and select **Create RESTful Service**.



3. In the **Create RESTful Service from Java Class** dialog box, change the Root Path input field text to **tut/department** to define the base URI to access the department source. click **Next**.



4. In the **Configure HTTP Methods** table (note that you may need to resize the dialog box to see the full label string), select the **getAll** entry.

The `getAll()` public method in `DepartmentService.java` accesses the EJB business service to query a list of departments from the Departments table in the HR schema and returns the result as an XML response.

Create RESTful Service - Step 3 of 4

Create RESTful Service From Java Class

Select the methods to be promoted as HTTP methods in the resource class. Also provide the required MIME type for the data content.

Root Path:

Consumes: ... Produces: ...

Configure HTTP Methods:

Name	Type	Consumes	Produces	Path
getAll				

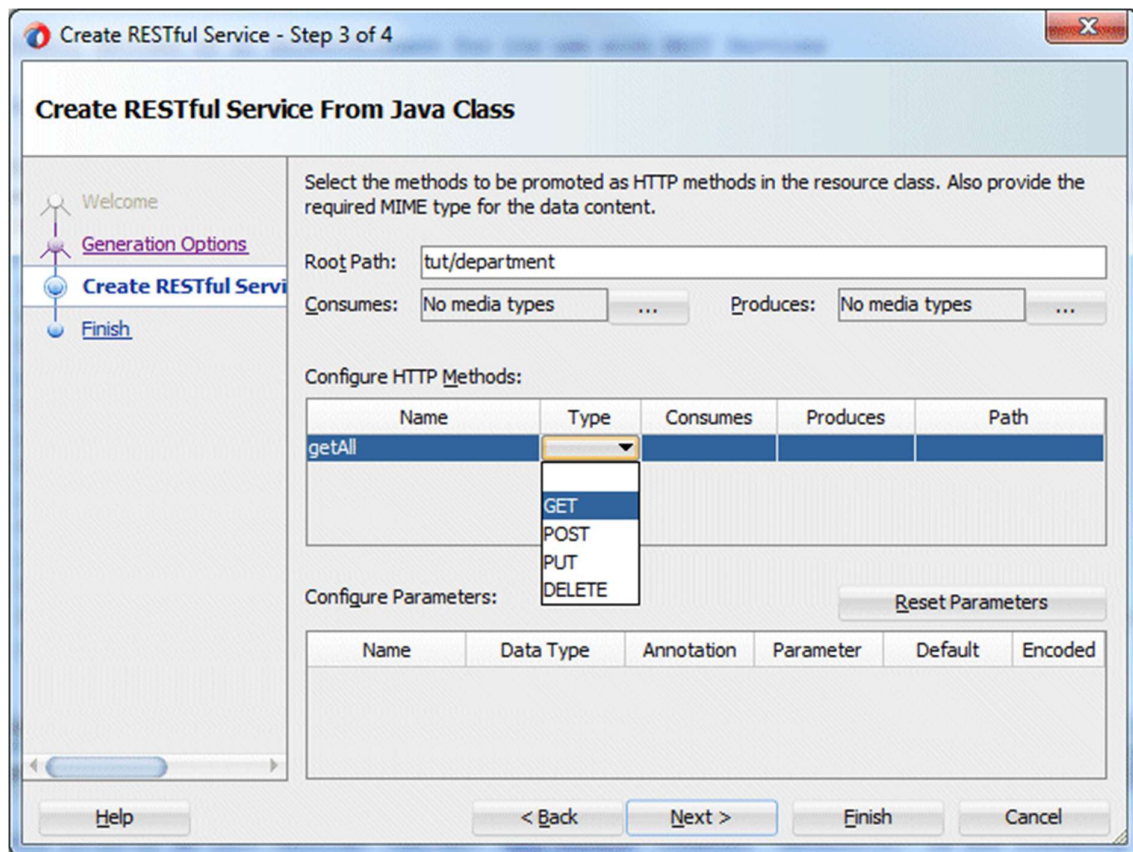
Configure Parameters:

Name	Data Type	Annotation	Parameter	Default	Encoded
------	-----------	------------	-----------	---------	---------

Help < Back Next > Finish Cancel

5. Because read requests in REST are Get requests, select **GET** from the **Type** column for the getAll entry.

No input arguments are required, so leave the Consumes field empty.

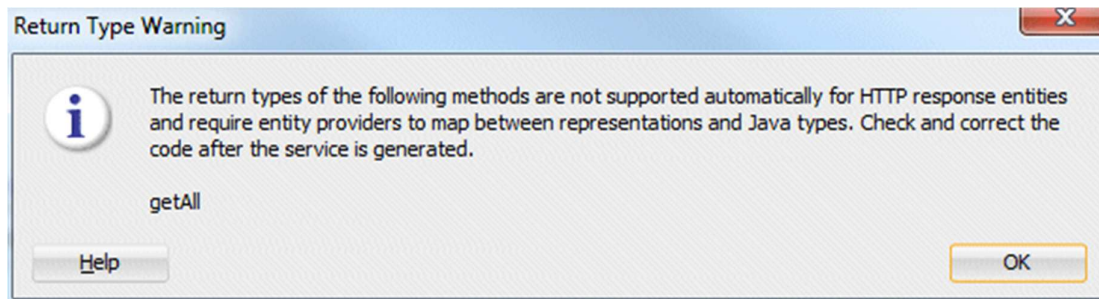


6. In the **Produces** field, click the ... button and select **application/xml**, and then click **OK**.

Configure HTTP Methods:

Name	Type	Consumes	Produces	Path
getAll	GET		1 media types	/

- Click **OK** in the **Return Type Warning** dialog box. The hands-on sample uses JAXB annotations added to the EJB entities to parse Java objects to XML and vice versa, so you can ignore this dialog box.



Click **Finish** to create the service.

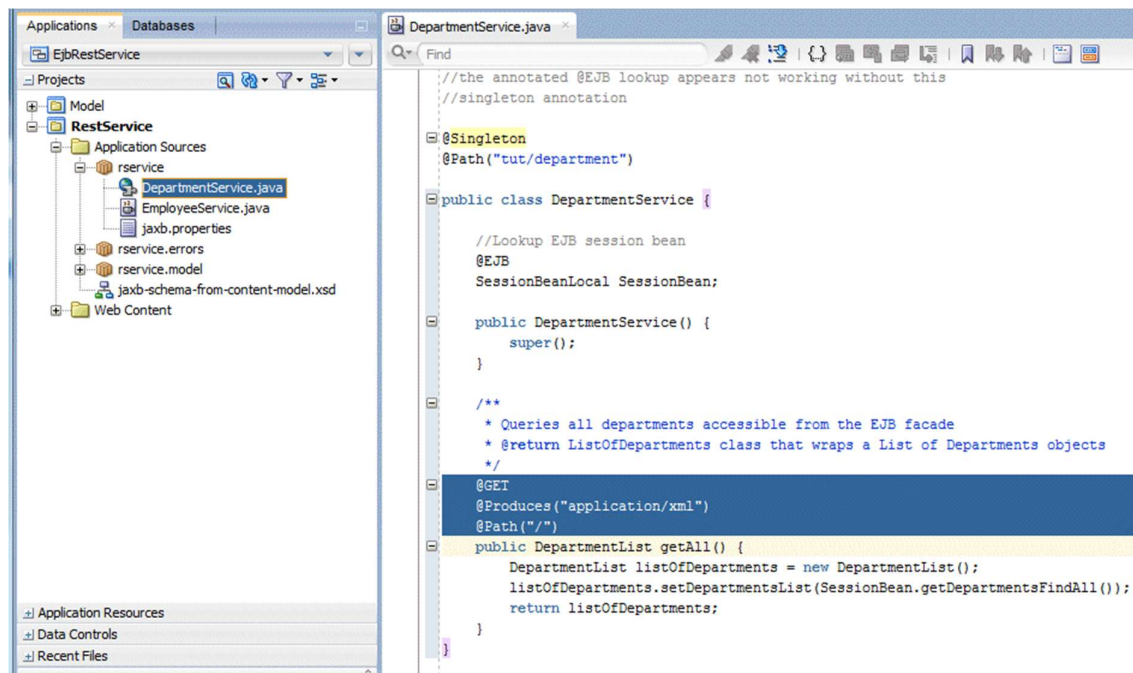
- Click **Finish** to create the service.

Oracle JDeveloper opens the DepartmentService.java class in the Java code editor. The class shows the getAll method annotated with

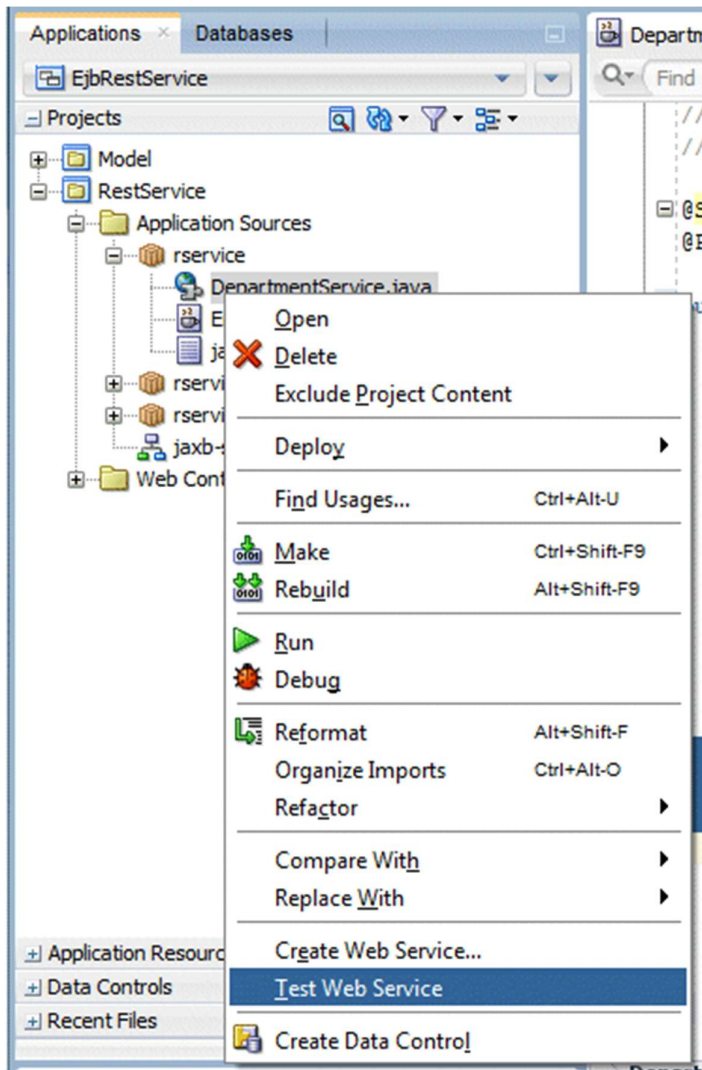
```
@GET
@Produces("application/xml")
@Path("/")
```

The DepartmentService class itself is annotated with @Path to identify the root URI for this REST resource:

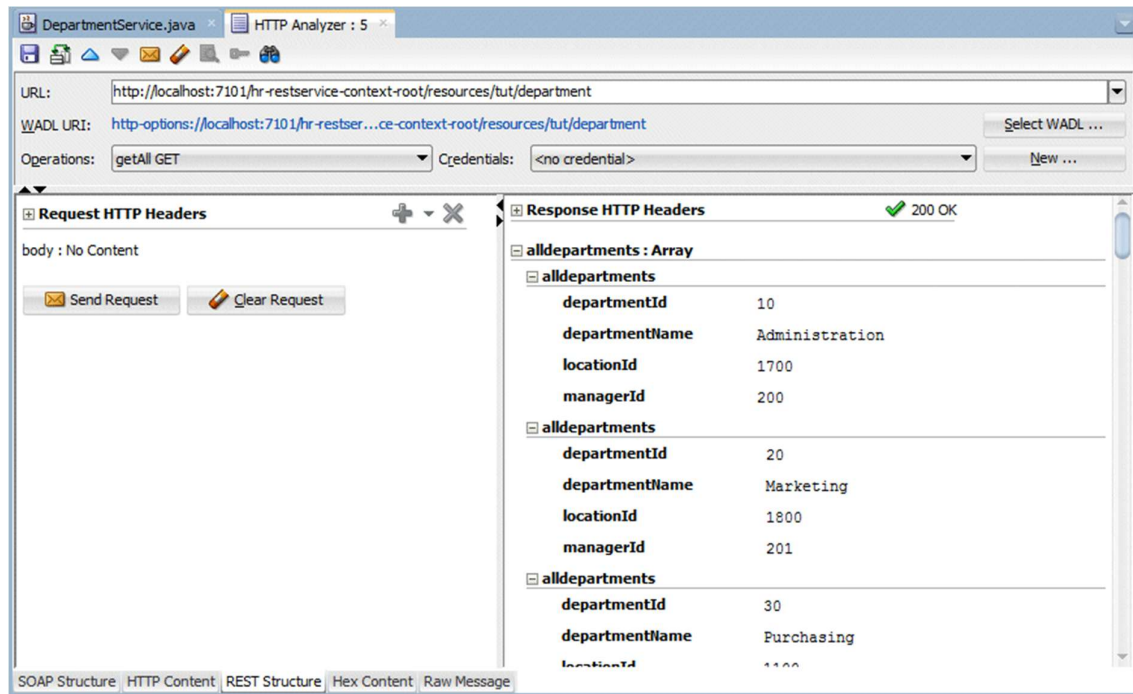
```
@Path("tut/department")
```



10. Right-click the **DepartmentService.java** file, and select **Test Web Service** to test the REST service in the HTTP Analyzer.



11. Click **Send Request** in the opened HTTP Analyzer window to issue a Get request for the REST web service base URI and obtain a list of departments

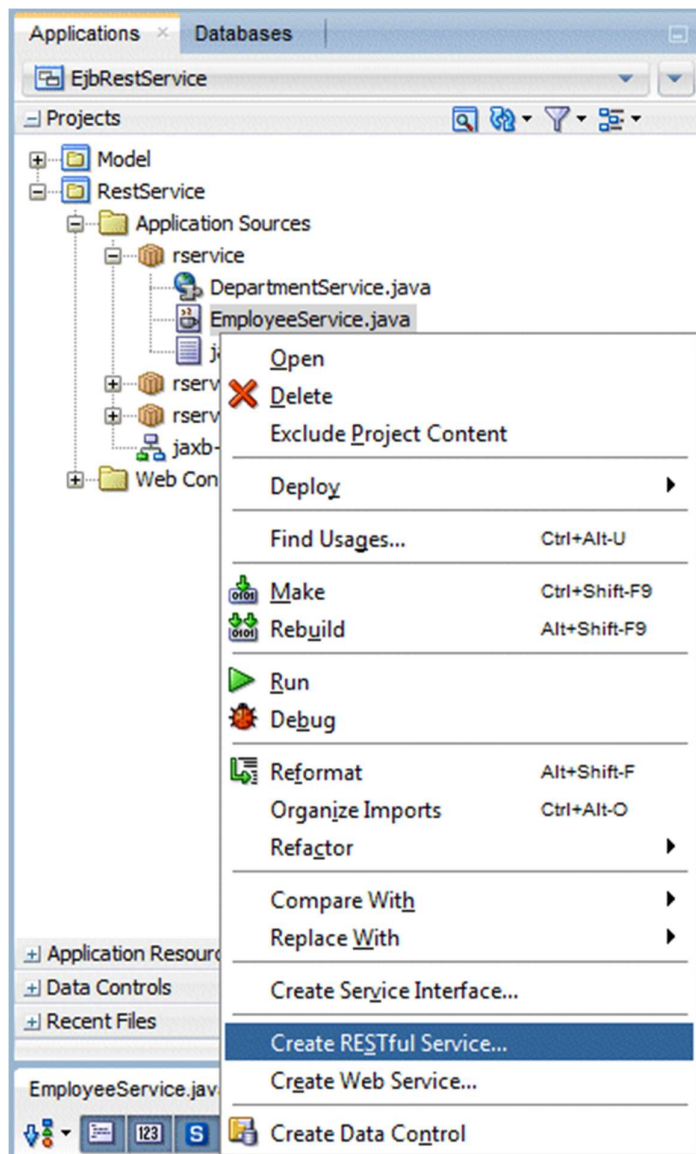


12. The log window shows a REST service base URL that looks similar to `http://127.0.0.1:7101/hr-restservice-context-root/resources/tut/department`. Make a note of the actual REST URL—you will need it later.

Step 3: Create the Employee REST Service

Next, configure the `EmployeeService` class as a REST service, which requires more configuration, because it includes support for update and delete operations.

1. In the **Application** window, in **RestService > Application Sources > rservice**, right-click the `EmployeeService.java` class and select **Create RESTful Service**.



2. In the **Create RESTful Service from Java Class** dialog box, change the **Root Path** input field to **tut/employee**.

Create RESTful Service - Step 3 of 4

Create RESTful Service From Java Class

Select the methods to be promoted as HTTP methods in the resource class. Also provide the required MIME type for the data content.

Root Path:

Consumes: ... Produces: ...

Configure HTTP Methods:

Name	Type	Consumes	Produces	Path
getByDepartmentId				
update				
removeById				

Configure Parameters:

Name	Data Type	Annotation	Parameter	Default	Encoded

Help < Back Next > Finish Cancel

- In the Configure HTTP Methods table, edit the methods as follows:

Name	Type	Consumes	Produces
getByDepartmentId	GET		application/xml
update	POST	application/xml	
removeById	DELETE		

4.

Create RESTful Service - Step 3 of 4

Create RESTful Service From Java Class

Select the methods to be promoted as HTTP methods in the resource class. Also provide the required MIME type for the data content.

Root Path:

Consumes: ... Produces: ...

Configure HTTP Methods:

Name	Type	Consumes	Produces	Path
getByDepartmentId	GET		1 media types	/{departmentId}
update	POST	1 media types		/
removeById	DELETE			/{employeeId}

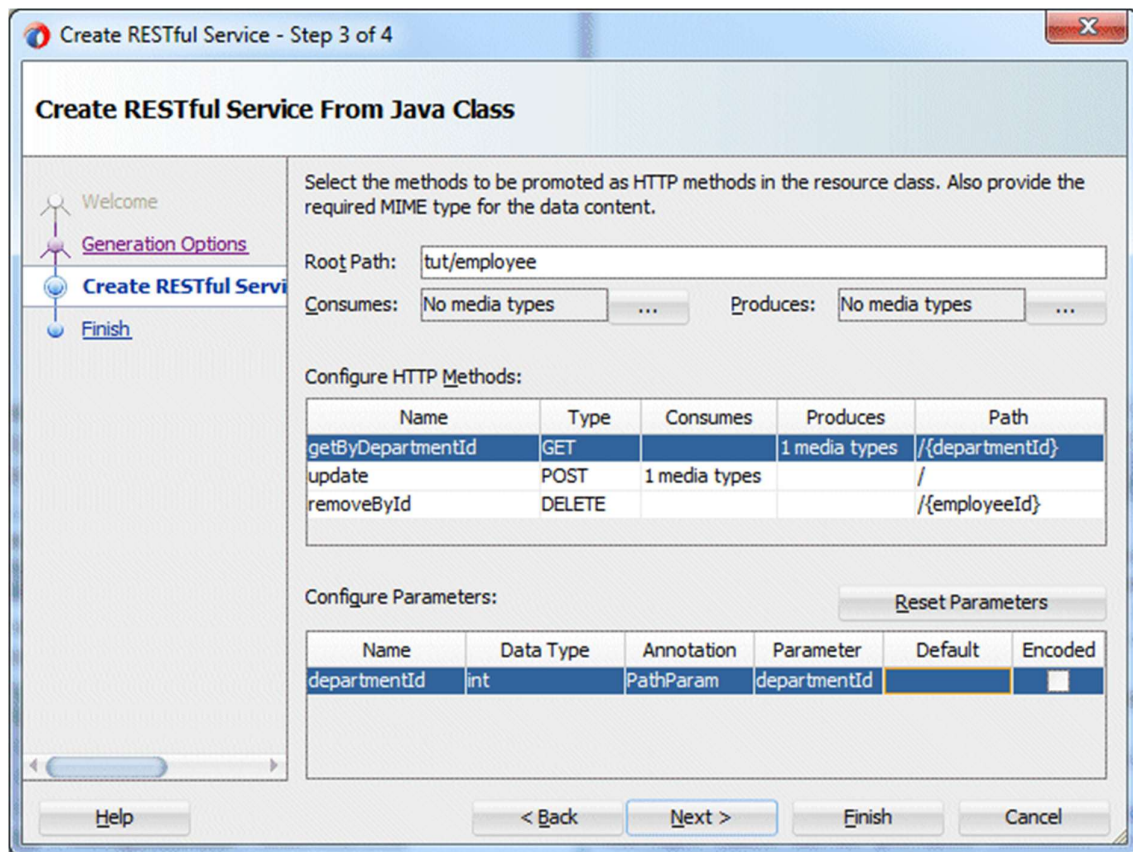
Configure Parameters:

Name	Data Type	Annotation	Parameter	Default	Encoded
departmentId	int				☐

Help < Back Next > Finish Cancel

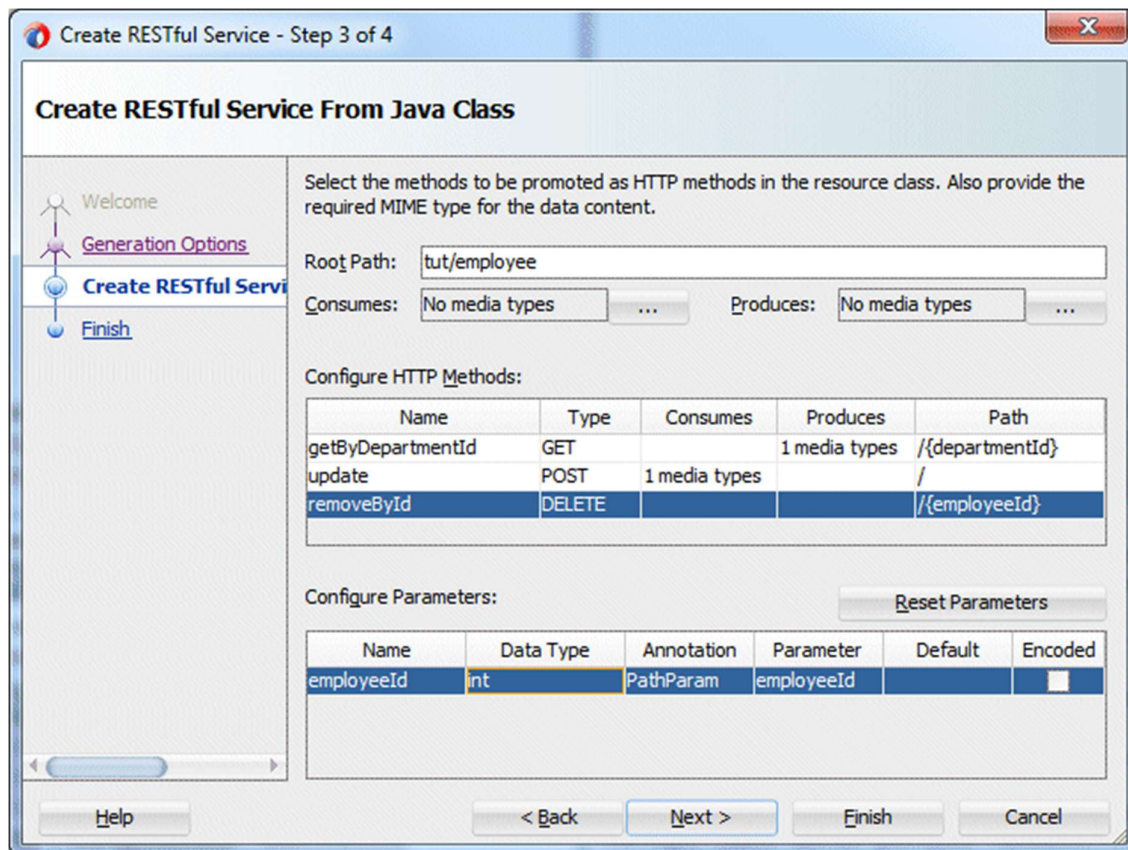
5. Next, select the **GetByDepartmentId** method and edit the **Configure Parameters** table as follows:

Name = departmentId
Data Type = int
Annotation = PathParam
Parameter = departmentId
Default = default<null>
Encoding = default<unselected>



- Next, select the **removeById** method and edit the **Configure Parameters** table as follows:

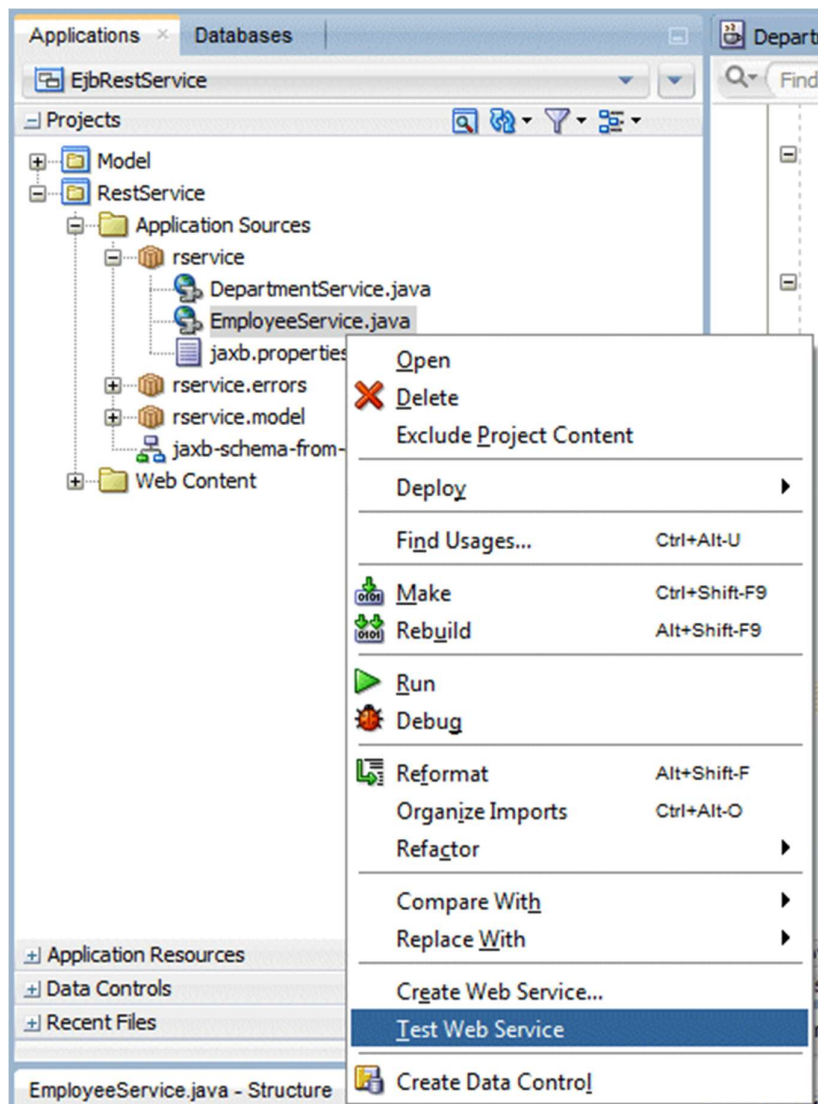
Name = employeeId
Data Type = int
Annotation = PathParam
Parameter = employeeId
Default = default<null>
Encoding = default<unselected>



- Click **Finish** and **OK** to confirm the Return Type Warning..

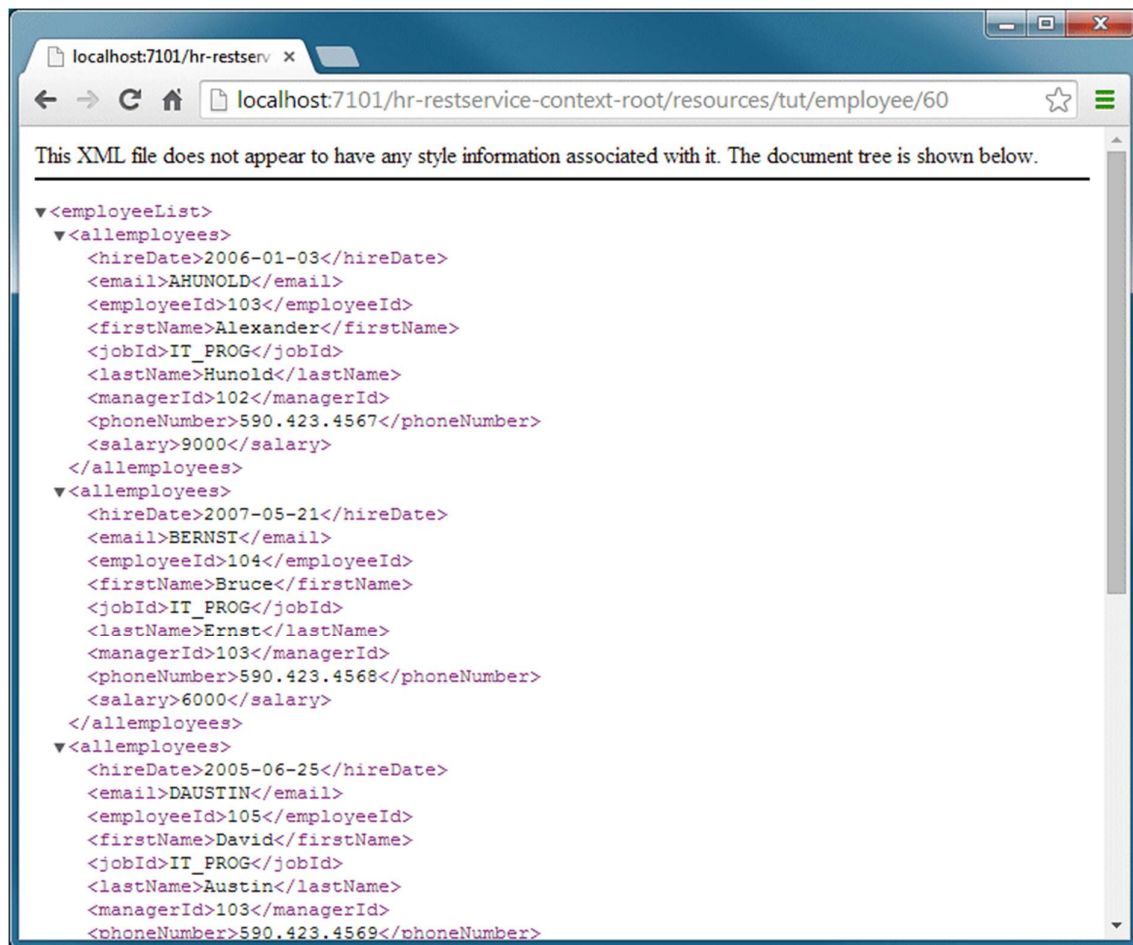


- To test the service, right-click the **EmployeeService** class and select **Test Web Service**.



However, rather than using the HTTP Analyzer to test the service, we'll check it from a browser. Open a browser window and copy/paste the URL (**<http://localhost:7101/hr-restservice-context-root/resources/tut/employee/>**) into it. Add a "60" to the end of the URL to find all the employees in department 60.

If you scroll down, you should see 5 employees: AHUNOLD, BERNST, DAUSTIN, VPATABAL and DLORENTZ.



9. Make a note of the URL, you'll use it later - <http://localhost:7101/hr-restservice-context-root/resources/tut/employee>.

You successfully configured the two JavaBeans in the `RestService` project as services. Each service represents a resource: a department or an employee. In this section, you performed a more complex configuration that included input arguments defined for methods.

The `getByDepartmentId` and `removeById` methods both required a single input argument. Adding `/ {name}` to the REST URI path adds an annotation that enables the request to reference a resource, such as a department, by a unique locator.

So the annotated method would know where to get the required input argument, you added the `PathParam` annotation with the name of the variable in the request. The name of the variable is the same as the variable name added to the REST path, such as `departmentId` in `/ {departmentId}`.

In the next section, you'll open a new application, create data controls from the REST services and develop a page that consumes the REST data controls.

Part 2: Developing the Client Application

In this section you will start creating a JSF page using data controls based on EJ/JPA components.

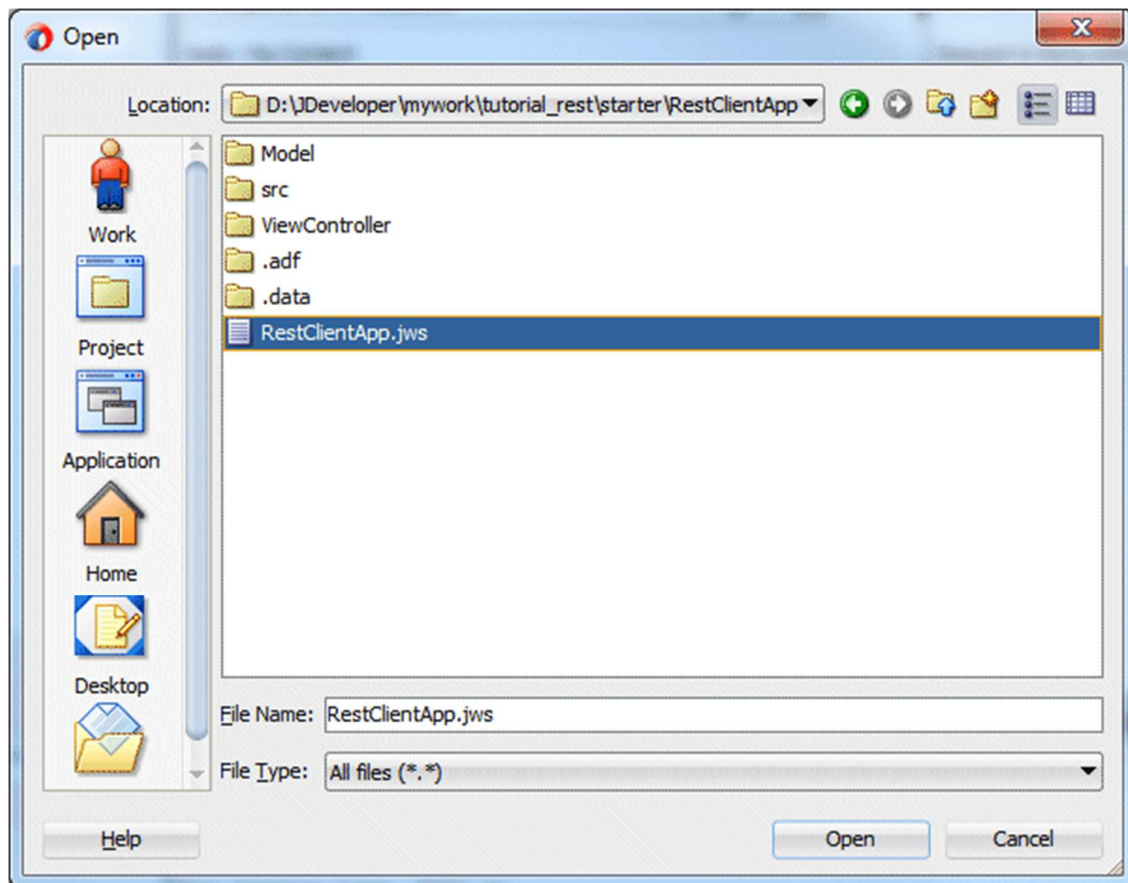
You can design a databound user interface by dragging an item from the Data Controls panel and dropping it on a page as a specific UI component. When you use data controls to create a UI component, JDeveloper automatically creates the various code and objects needed to bind the component to the data control you selected.

Step 1: Create the Department REST Data Controls

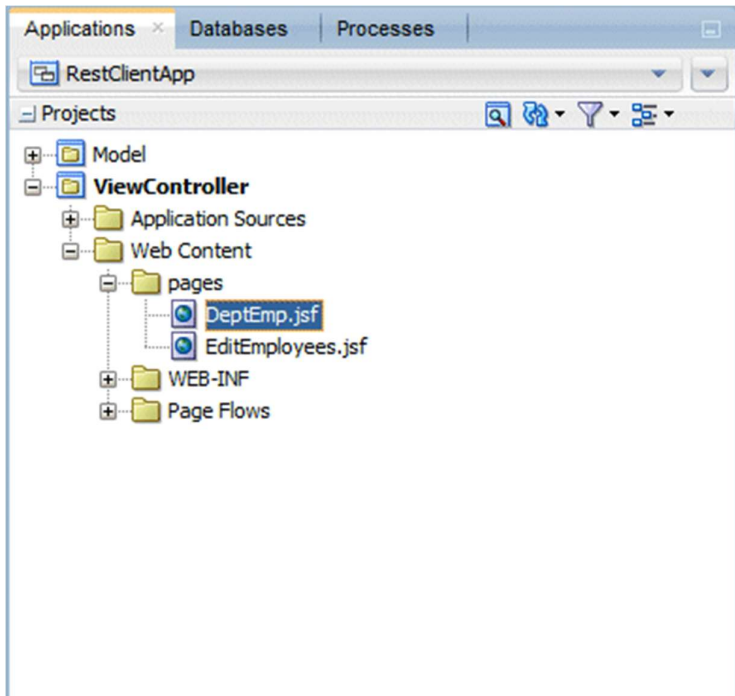
In the next few steps, you build data controls for the Department Service. The REST services are consumed in Oracle ADF by use of the new REST service data control (REST DC). To create the REST DC model.

You'll create a REST data control for the department resource. The data control exposes a single method for the service GET request and expects an XML response to represent a list of departments.

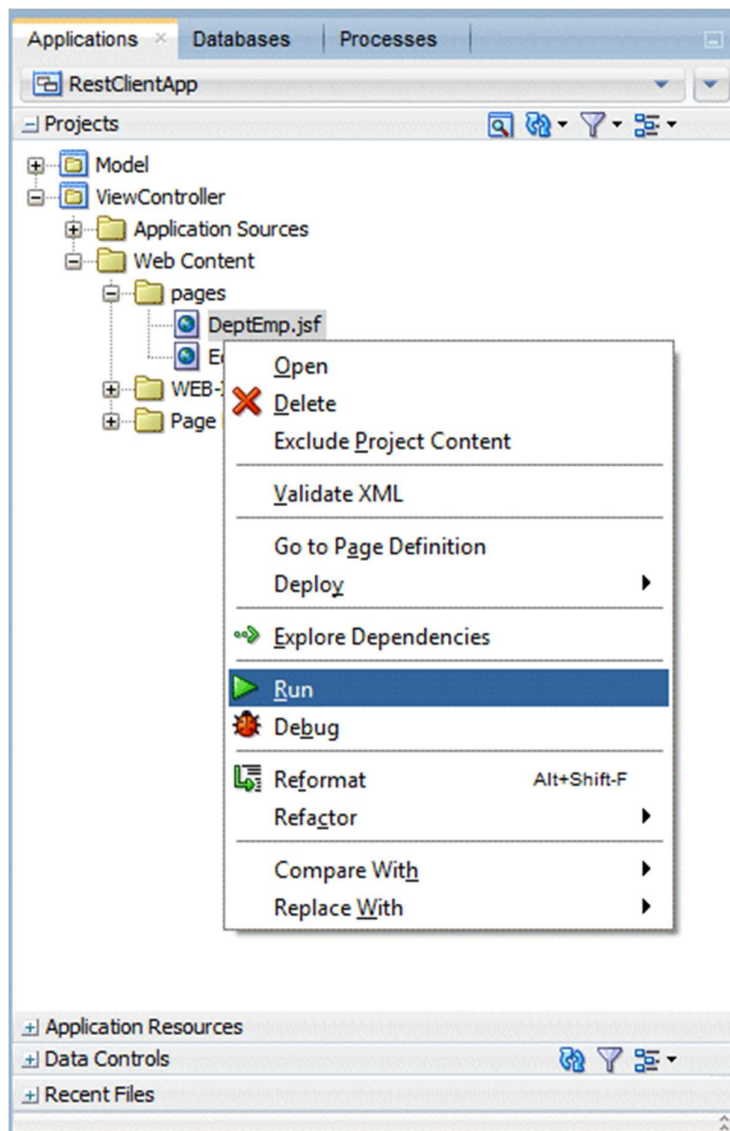
1. In Oracle JDeveloper, select **File > Open** and navigate to the directory containing the unpacked sample application zip file content. Open the **RestClientApp** folder and select the **RestClientApp.jws** file.



2. In Select the **Model** project, and select **File > New > From Gallery**, then expand the **Business Tier > Web Services** and select **Web Service Data Control (SOAP / REST)**. Then click **OK**.



3. In the **Create Web Service Data Control** dialog box, enter **DepartmentRestDC** in the **Name** field and select the **REST** radio button.



4. Click the green plus (+) icon next to the Connection field.

In the **Create URL Connection** dialog box, enter **DepartmentsRestConn** in the **Name** field.

Enter **http://127.0.0.1:7101/hr-restservice-context-root/resources/tut/department** in the **URL Endpoint** field.

Note: The URL Endpoint URL does not have an ending slash.

Click the **Test Connection** button to confirm the serve is available. If it is not, then go back into the EjbRestService and test the DepartmentService.

Create URL Connection

Configure a new URL connection. Choose Application Resources to add this connection to the current application. Choose Resource Palette if you don't want a single application to own it.

Create Connection in: ☒ Application Resources ☐ IDE Connections

Name:
DepartmentsRestConn

URL Endpoint:
http://127.0.0.1:7101/hr-restservice-context-root/resources/tut/department

Authentication Type:
None

Username:

Password:

Realm:

Test Connection

Status:
URL Connection Successful.

Help OK Cancel

5. Then click **OK** and then click Next in the Create Web Service Data Control dialog box.

Create Web Service Data Control - Step 1 of 5

Data Source

Data Source

Data Control Operations

Response Format

Endpoint Authentication

Finish

Name: DepartmentRestDC

☐ SOAP ☒ REST

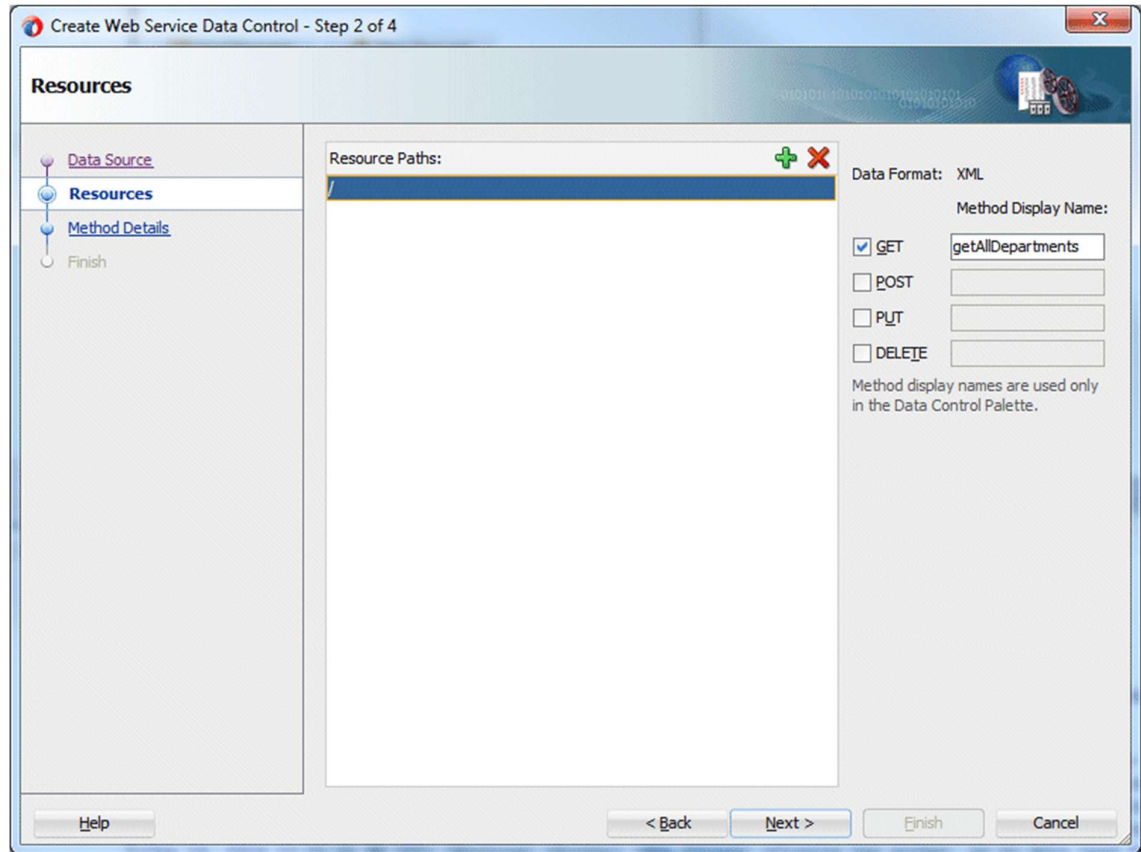
Select a URL connection to use for the Data Control.

Connection: DepartmentsRestConn

Source URI: http://127.0.0.1:7101/hr-restservice-context-root/resources/tut/department

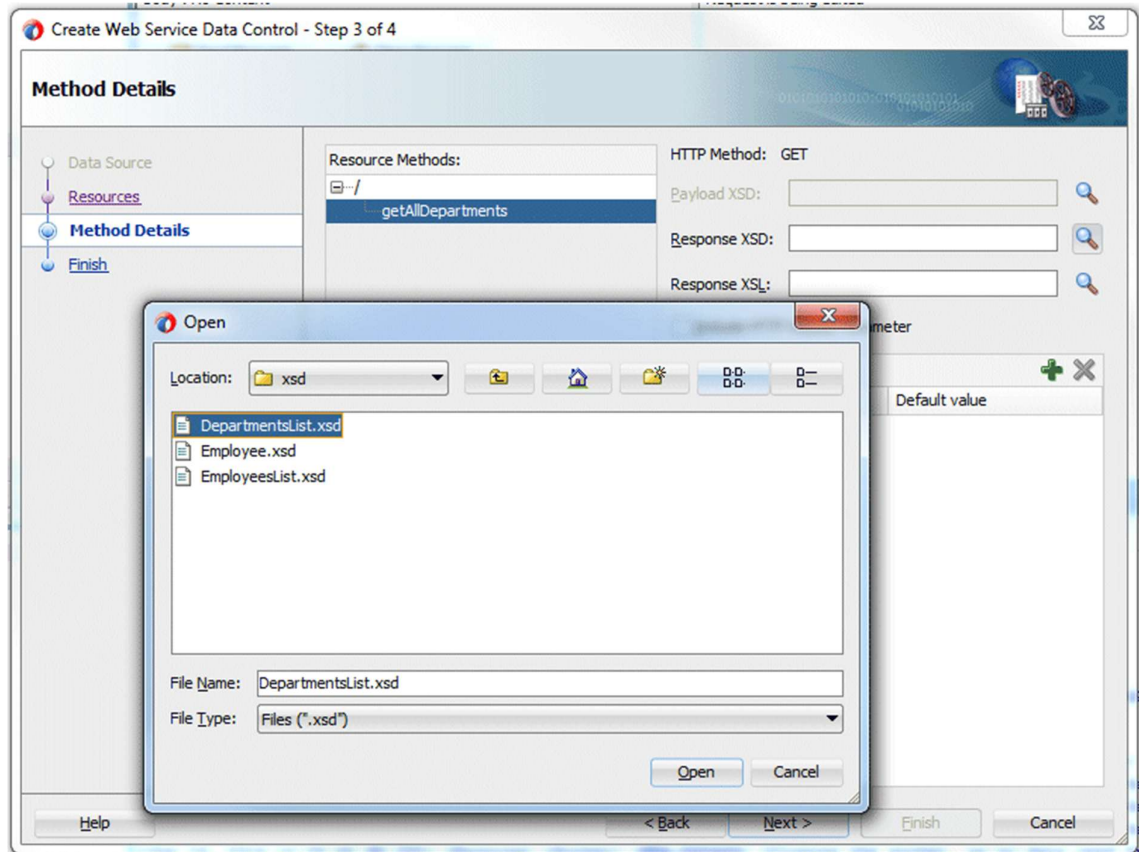
Help < Back Next > Finish Cancel

6. In Click the green plus (+) icon next to the **Resource Paths** label.
Change the **Resource Paths** value to /, and select **GET**.
Enter **getAllDepartments** as the **Method Display Name**.
Click **Next**.

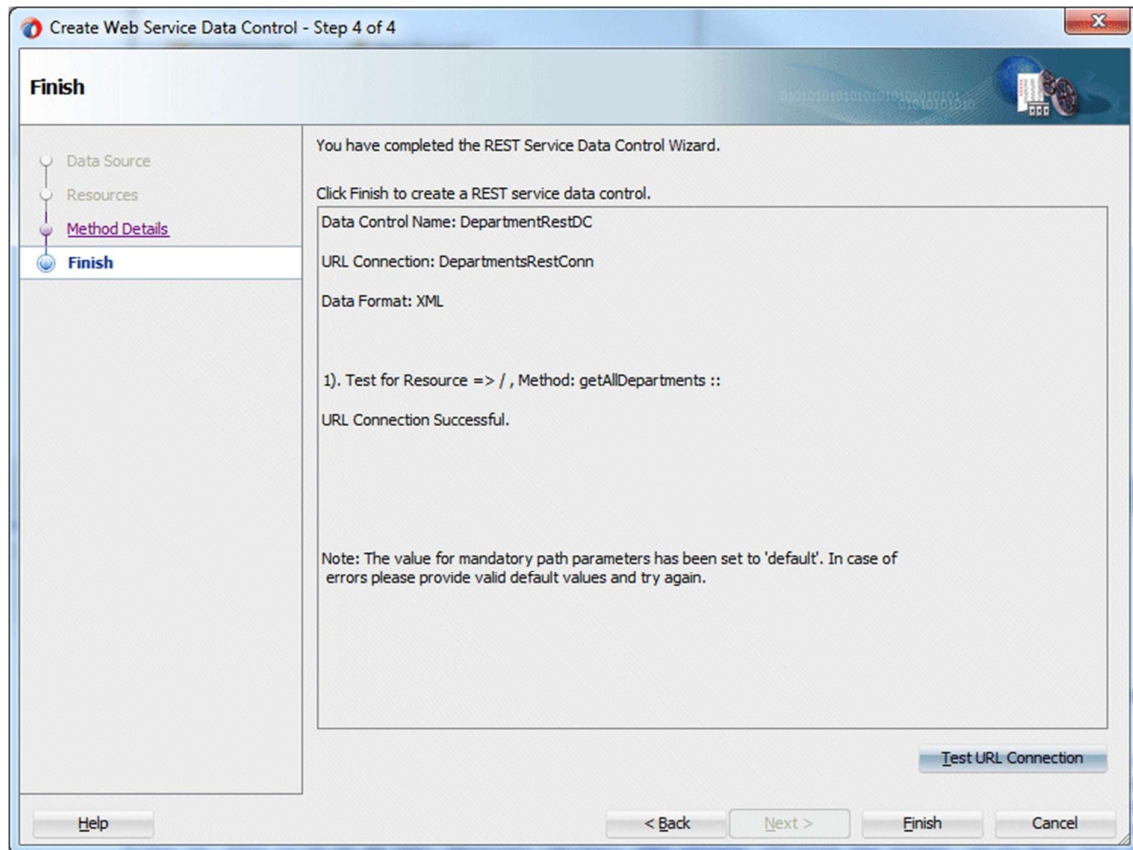


7. Select **getAllDepartments** under Resource Methods, and click the browse button (magnifier icon) next to the **Response XSD** field.

In the dialog box, open the **xsd** folder, select the **DepartmentsList.xsd** file, and click **Open**.



8. Click Next and then **Test URL Connection** button. Then **Finish** to create the data control.

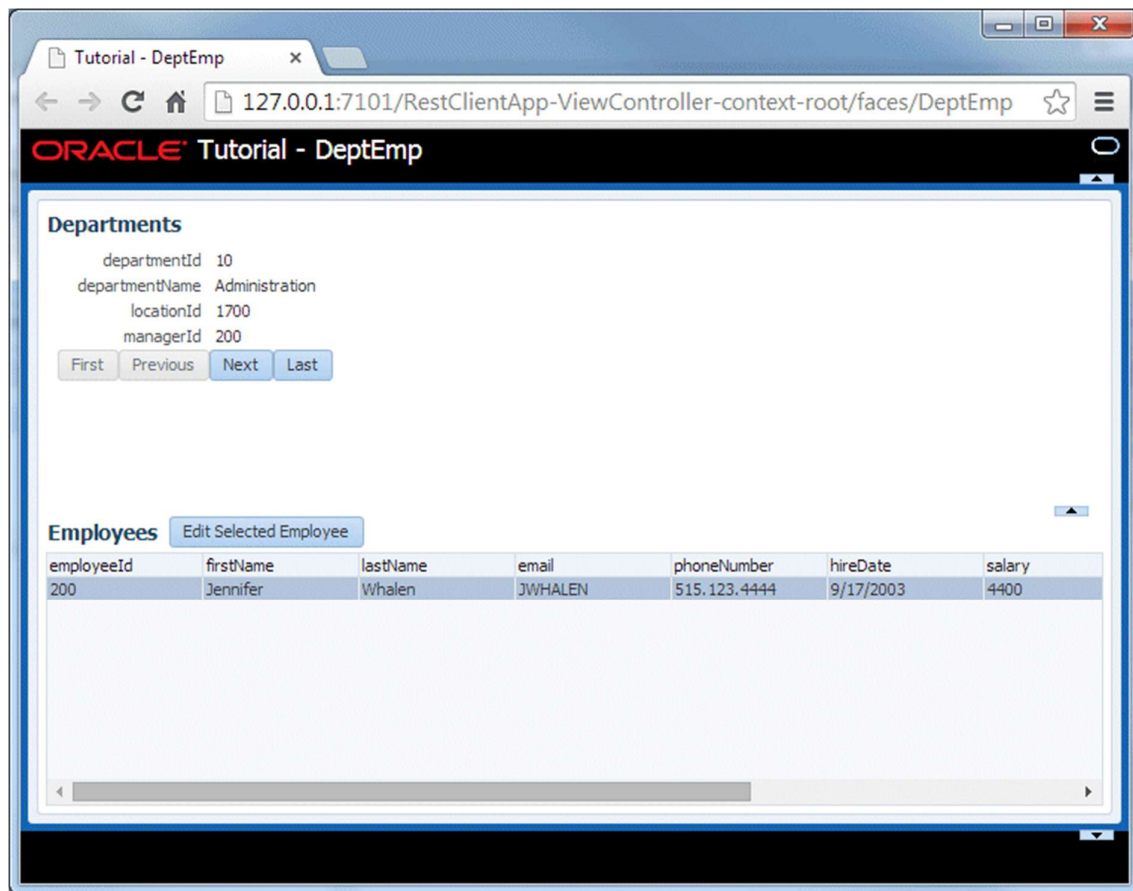


9. Click the **Save all**  icon to save your work.

Step 2: Create the Employee REST Data Controls

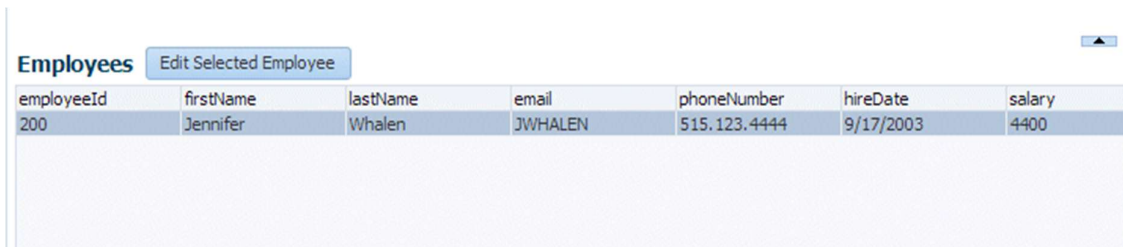
In the next few steps, you create a second data control. The employee REST data control you will build next is a bit more complex than the department data control, because it exposes three REST HTTP methods—for GET, POST, and DELETE—and also needs to define input arguments for GET and DELETE.

1. Again, in the Oracle JDeveloper Application window, right-click the **Model** project and select **New > Web Service Data Control (SOAP / REST)**.



2. In the **Create Web Service Data Control** dialog box, enter **EmployeeRestDC** in the **Name** field.

Select the **REST** radio button.



3. Click the green plus (+) icon next to the **Connection** field.
In the **Create URL Connection** dialog box, enter **EmployeesRestConn** in the **Name** field.
Enter **http://127.0.0.1:7101/hr-restservice-context-root/resources/tut/employee** in the **URL Endpoint** field.

Then click the **Test Connection** button.

Oracle Magazine Mar/Apr x

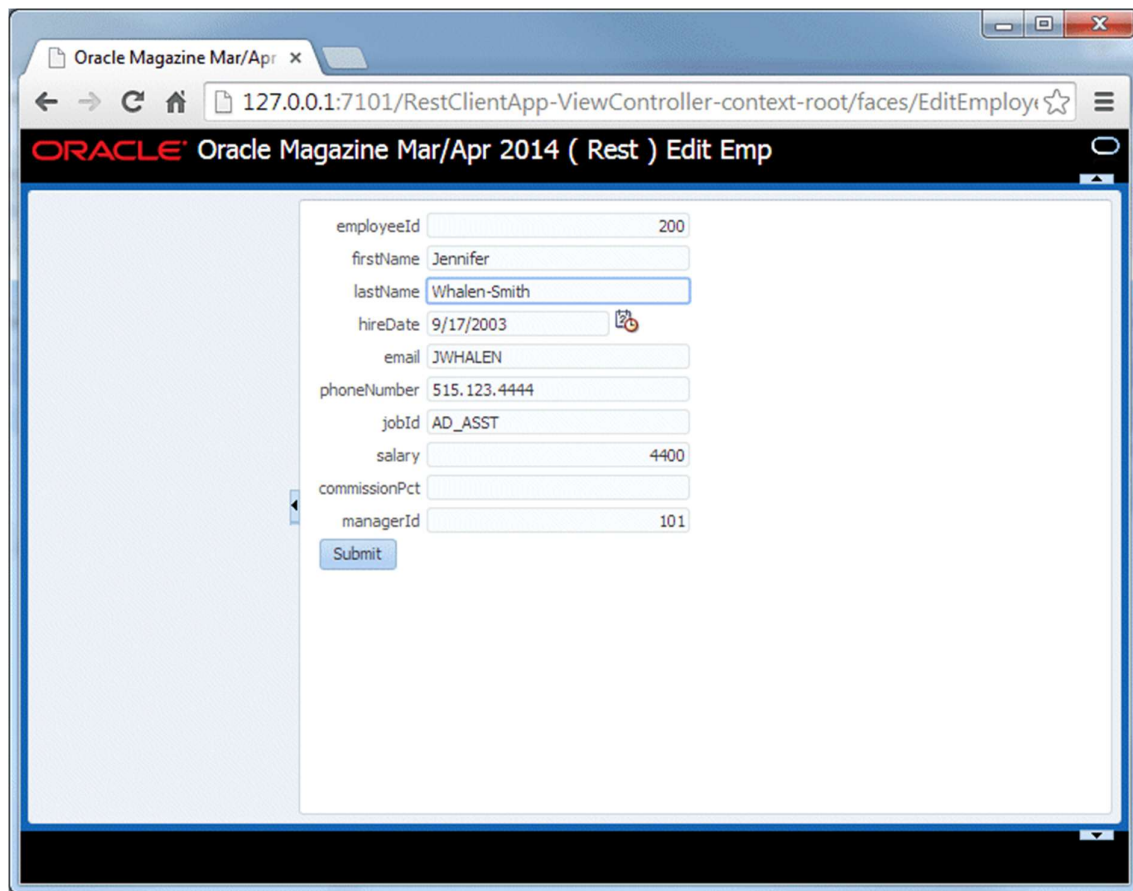
127.0.0.1:7101/RestClientApp-ViewController-context-root/faces/EditEmployee

ORACLE Oracle Magazine Mar/Apr 2014 (Rest) Edit Emp

employeeId	200
firstName	Jennifer
lastName	Whalen
hireDate	9/17/2003
email	JWHALEN
phoneNumber	515.123.4444
jobId	AD_ASST
salary	4400
commissionPct	
managerId	101

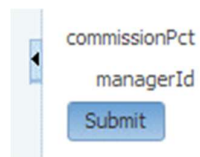
Submit

4. Click **OK**, then **Next**.

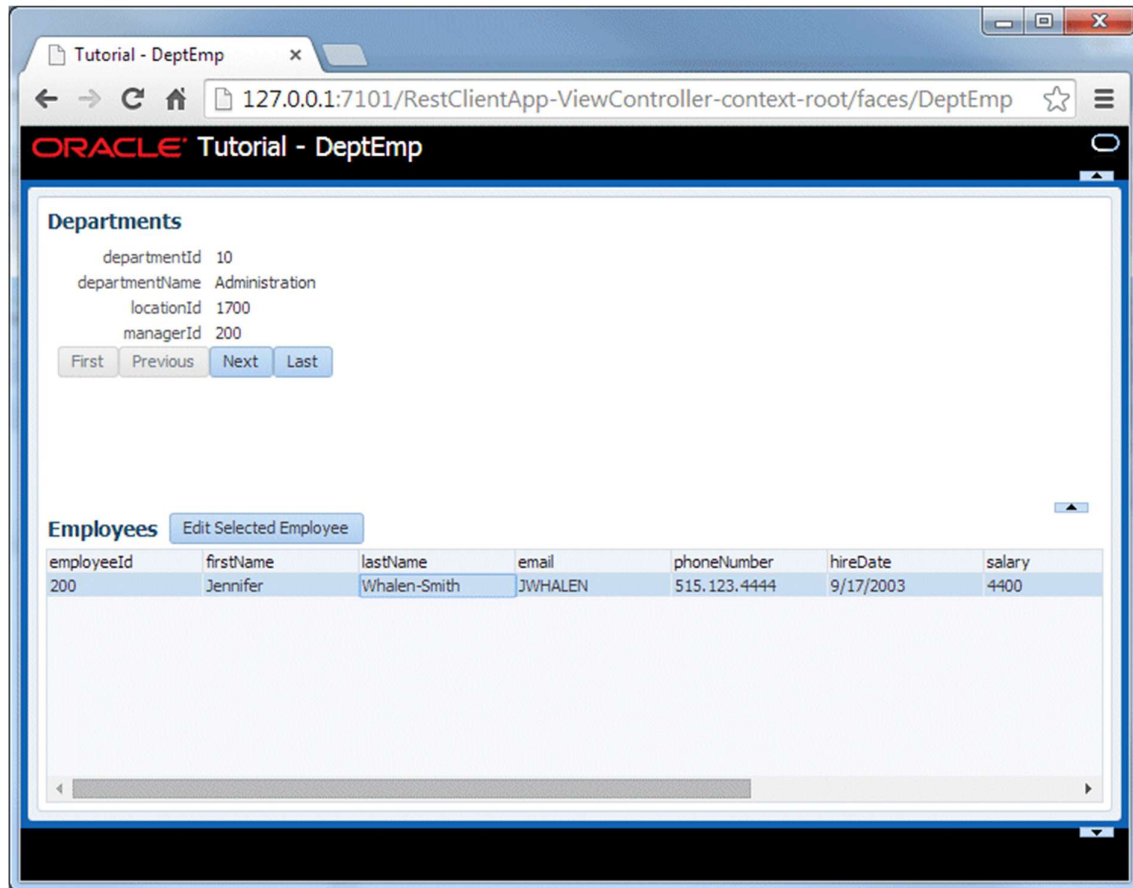


5. Click the green plus (+) icon next to the **Resource Paths** label.
Change the **Resource Paths** value from /path0 to /##departmentId##, and select **GET**.
Enter **getEmployeesByDepartment** as the **Method Display Name** value for GET.

Note: The ##departmentId## flag defines parameters in the Oracle ADF REST data control.
The parameter names will show as arguments to the methods in the data control panel.

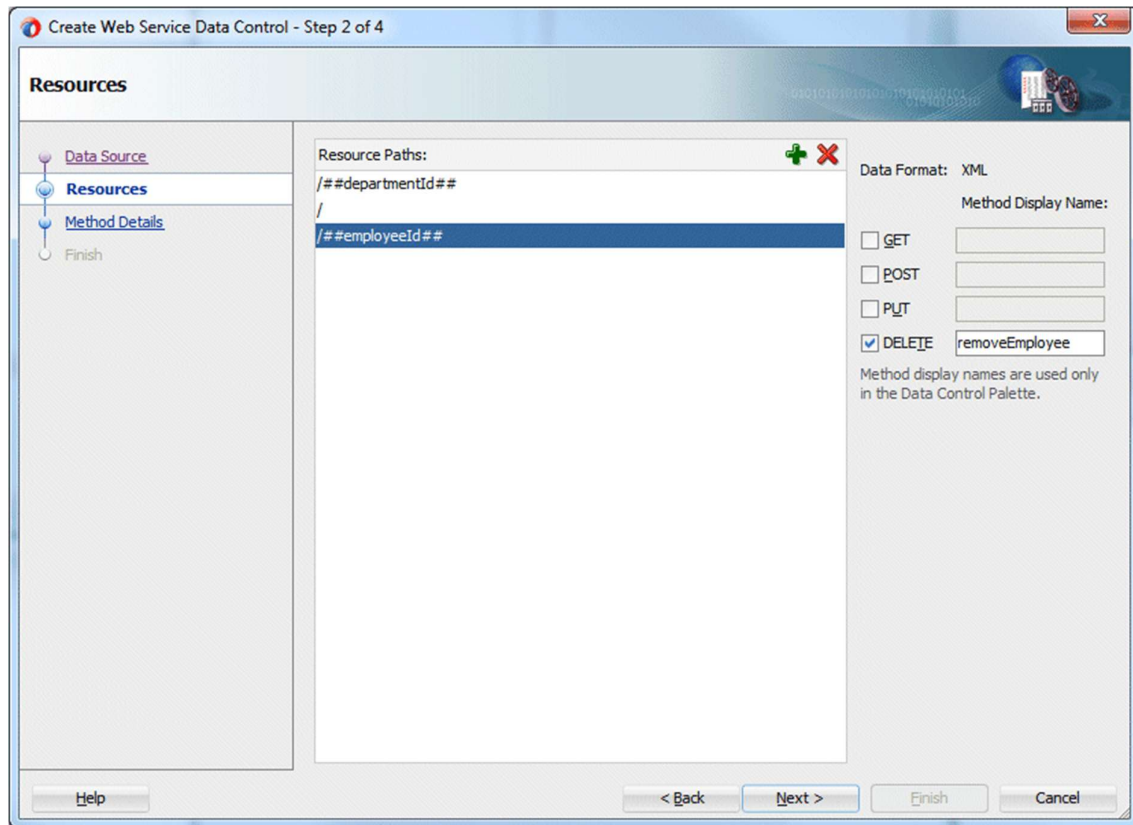


6. Again, click the green plus (+) icon next to the **Resource Paths** label.
Change the **Resource Paths** value from /path0 to /, and select **POST**.
Enter **updateEmployee** as the value of **Method Display Name** for POST.

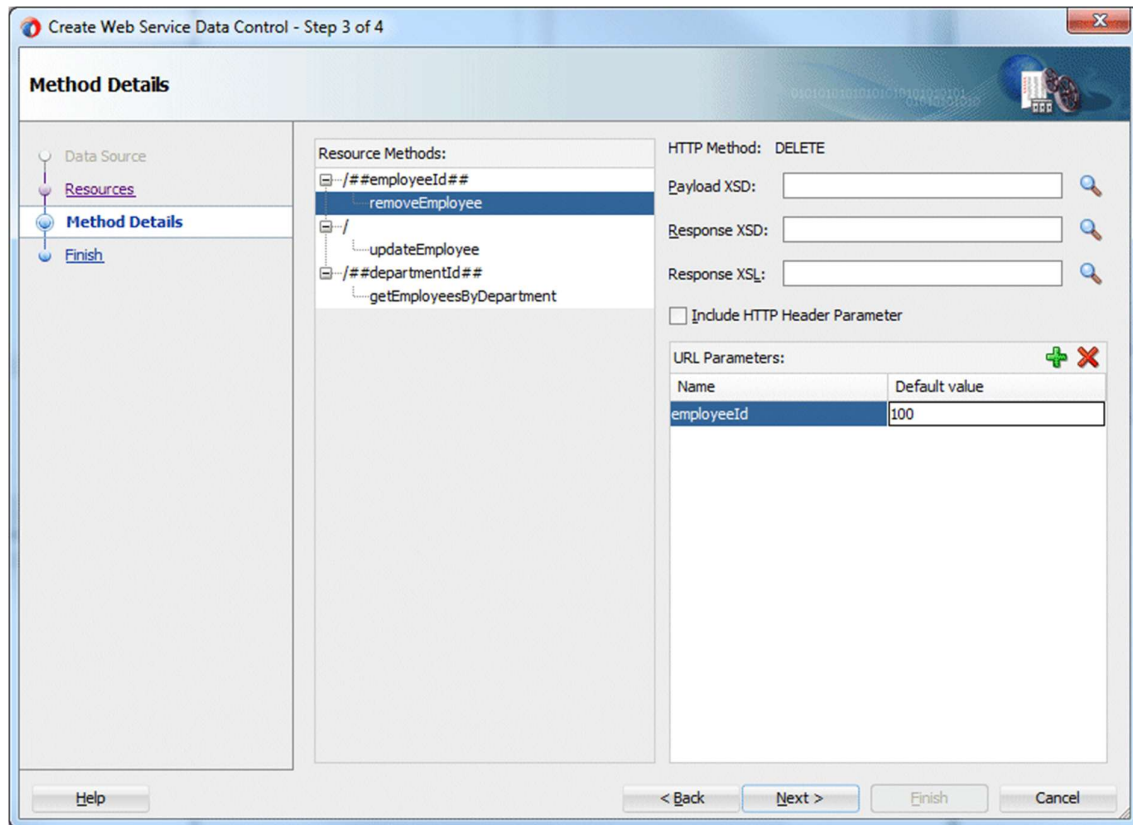


- Again, click the green plus (+) icon next to the **Resource Paths** label. Change the **Resource Paths** value from /path0 to /##employeeId##, and select **DELETE**. Enter **removeEmployee** as the value of **Method Display Name** for DELETE.

Then click **Next**.



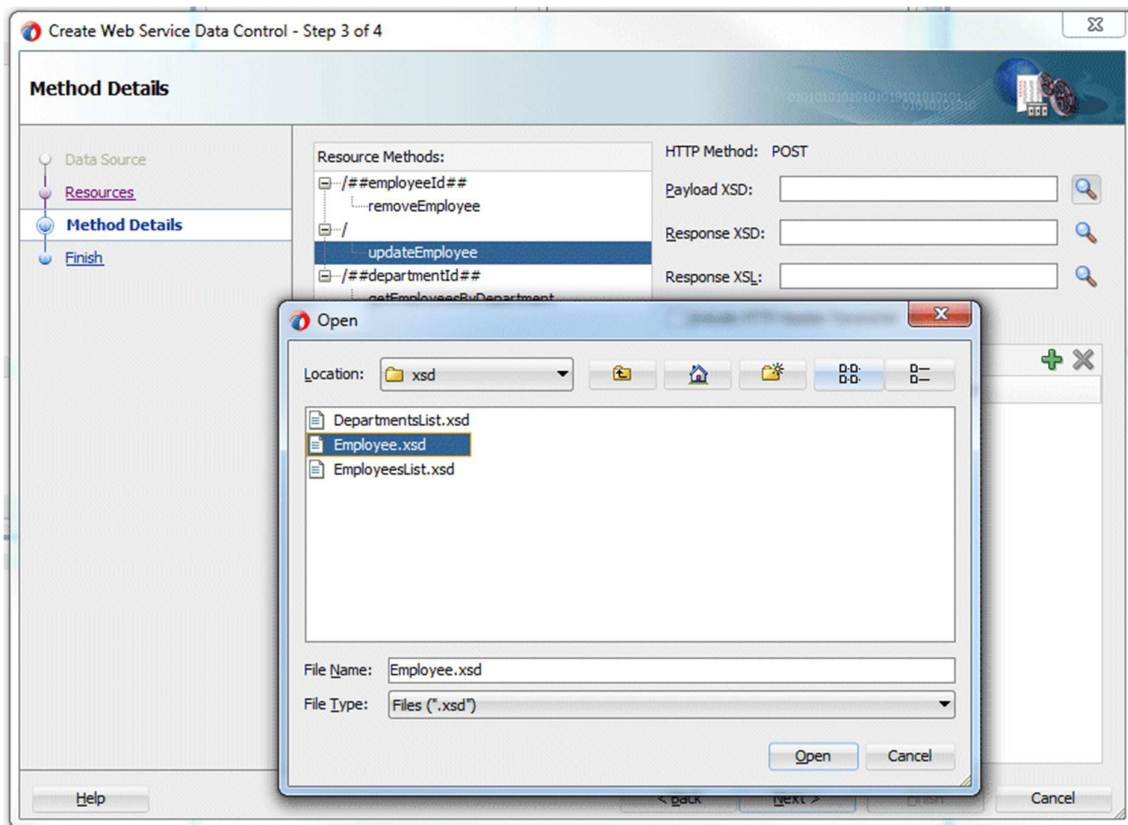
8. Select **removeEmployee** from the Resource Methods list, and enter a default value of **100** for **employeeId** (in the URL Parameters section).
Note: The 100 value is used to verify that the REST method request is valid at design time. (No delete will be processed, however) .



9. Select **updateEmployee** from the Resource Methods list, and click the browse button (magnifier icon) next to the **Payload XSD** field.

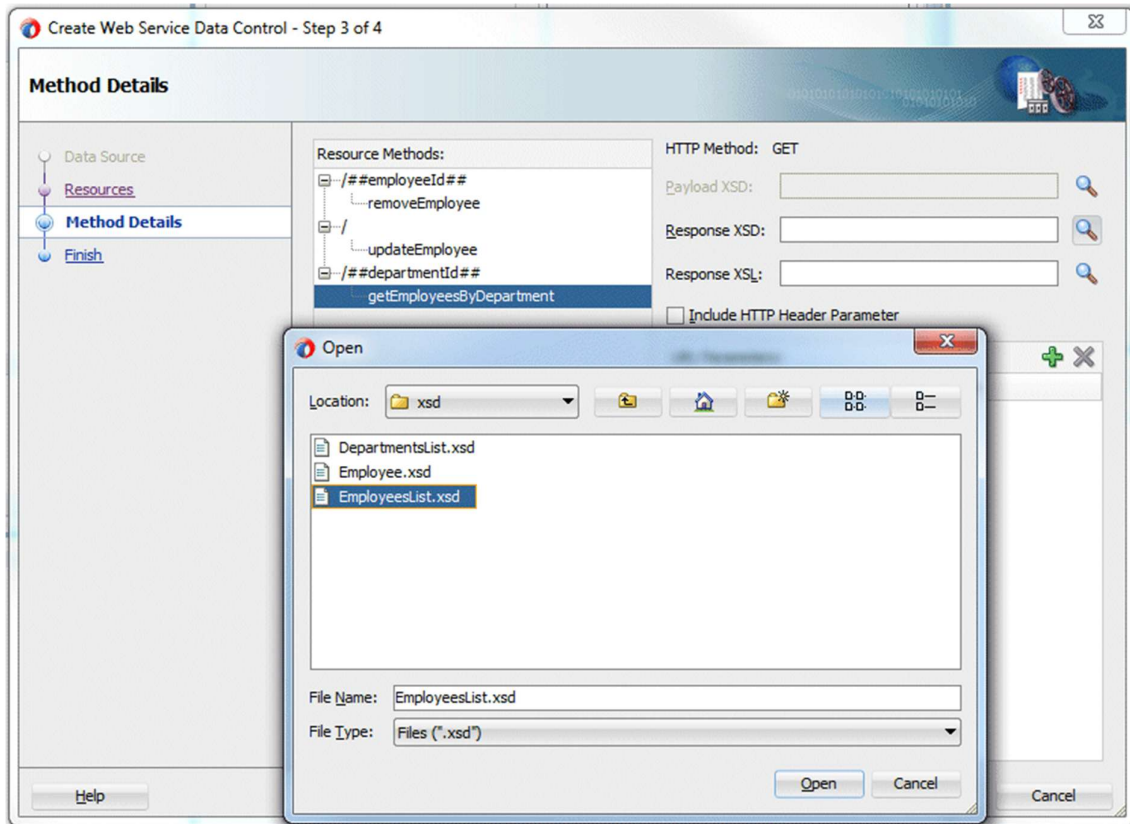
In the **Open** dialog box, open the **xsd** folder, select the **Employee.xsd** file, and click **Open**.

Note: The update method of the REST service has a single object argument of type Employee. Because this type is defined as a JAXB object, it can be marshaled and unmarshaled to and from XML. Therefore, you'll provide the XML representation of the argument.

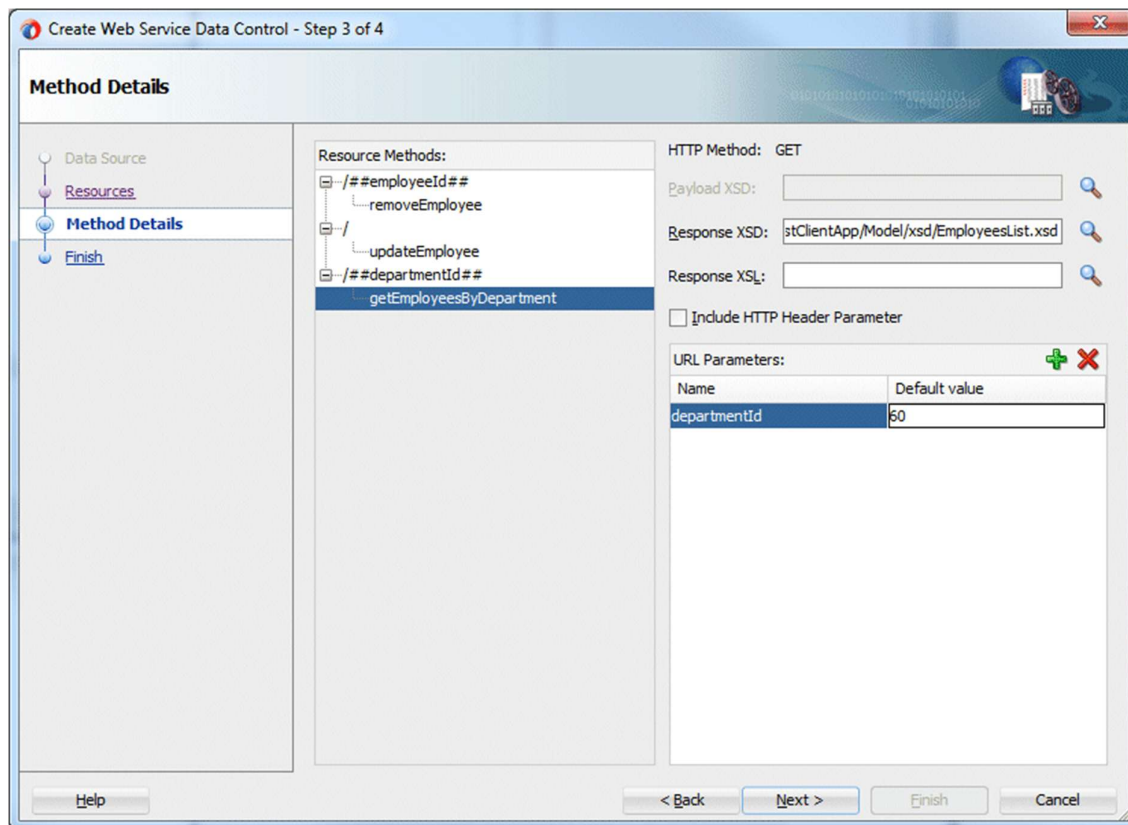


10. Select **getEmployeesByDepartment** from the Resource Methods list and click the browse button (magnifier icon) next to the **Response XSD** field.

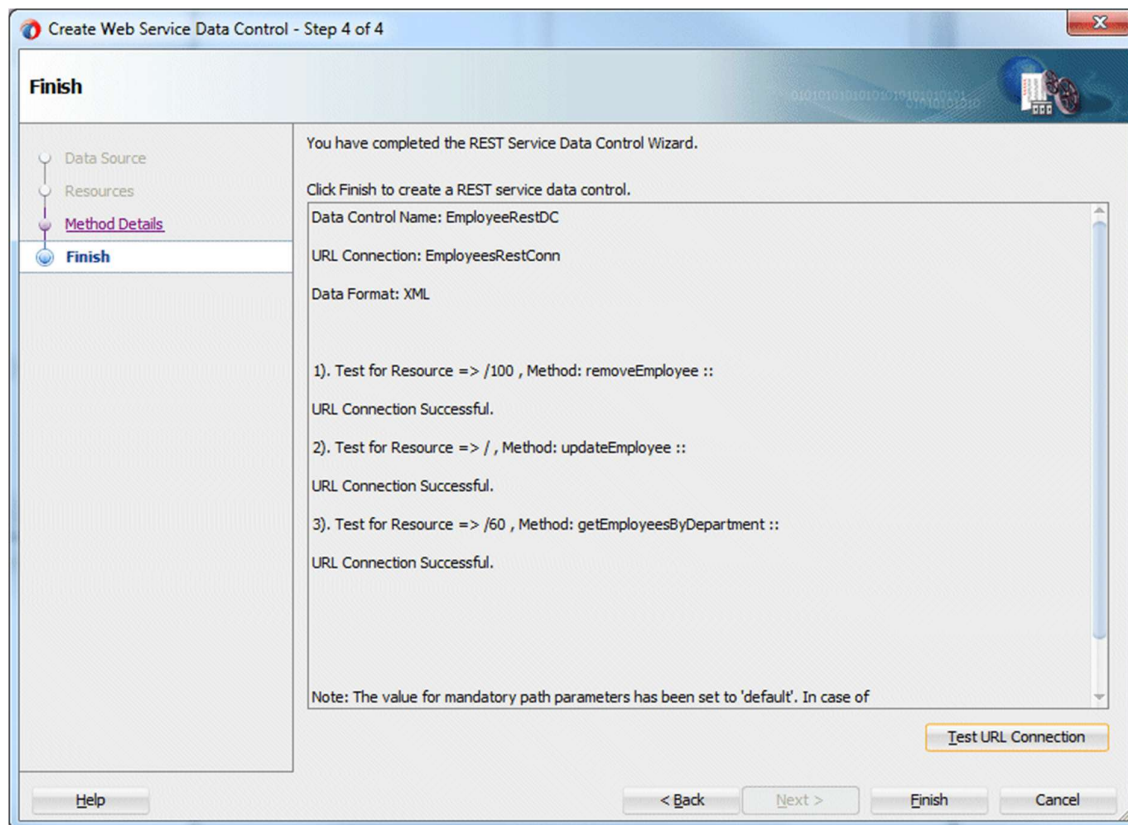
In the Open dialog box, open the **xsd** folder, select the **EmployeesList.xsd** file, and click **Open**..



11. Enter a default value of **60** for **departmentId** (in the URL Parameters section). Again, this is used for REST URI verification only at design time.

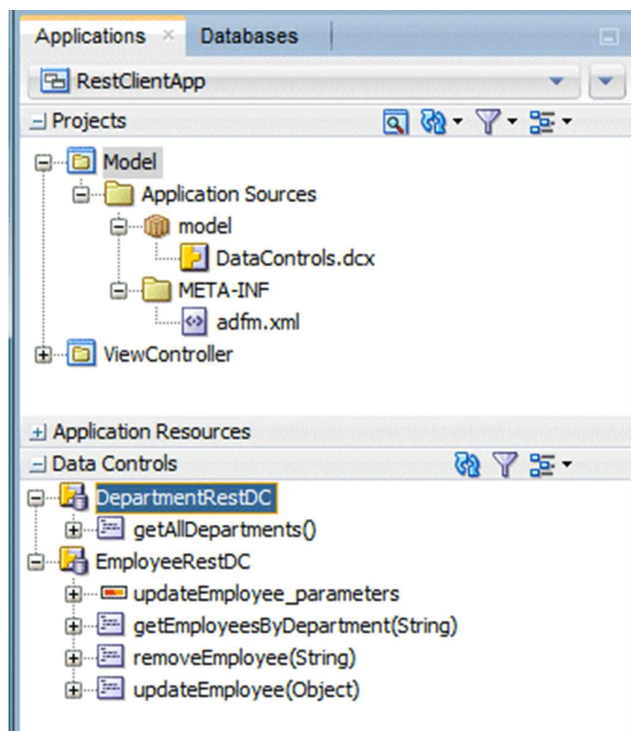


12. Click **Next**, and then click **Test URL Connection**.



13. Click **Finish** when the test is successful. Otherwise, check and correct the provided REST URI.

In the **Application** window, expand the **Data Controls** panel and refresh it by clicking the refresh button (icon with two arrows). The two data controls are now displayed. .



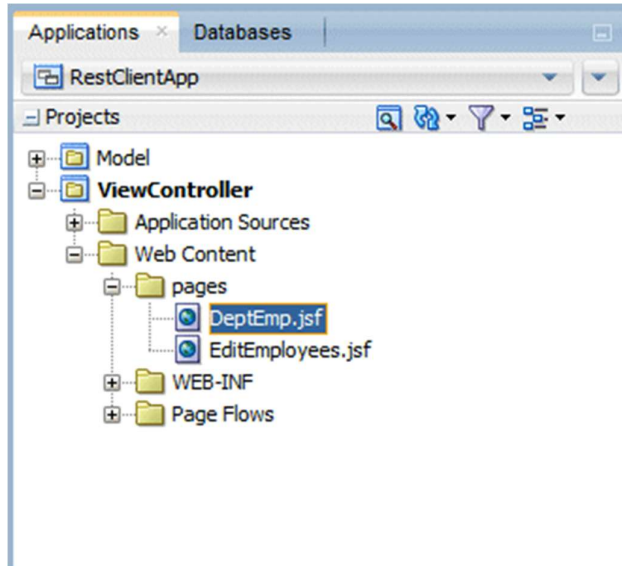
14. Click the **Save all**  icon to save your work.

Step 3: Build a Master-Detail Page

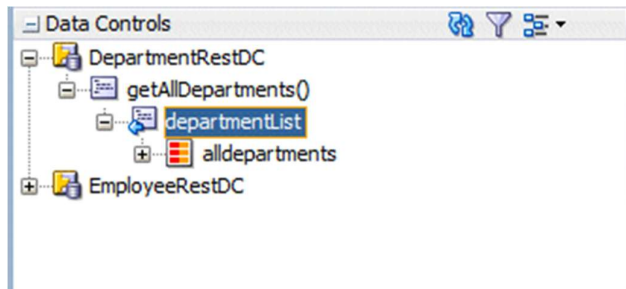
1. With the REST services and the REST data controls created, the final step is to build a master-detail page. To create the master-detail page,

In the Application window, expand the **ViewController** project and the **Web Content > pages** node.

Double-click the **DeptEmp.jsf** page to open it in the visual page editor.

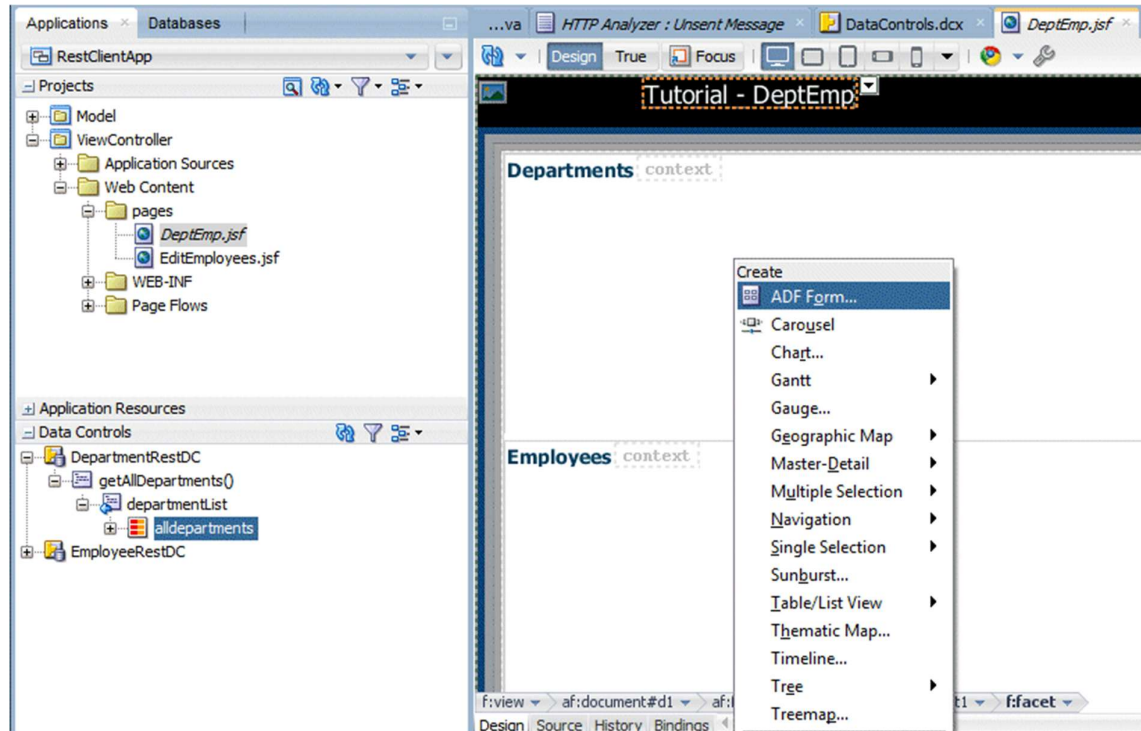


2. Expand the **Data Controls** panel and then the **DepartmentRestDC > getAllDepartments() > departmentList** nodes.



3. Drag the **alldepartments** entry from the Data Controls panel, and drop it in the **Departments** area in the visual page editor.

Select **ADF Form** from the Create menu.



4. Select the **Read-Only Form** checkbox in the **Create Form** dialog box.
In the **Include Controls for** section, select **Row Navigation** and leave all the field at their defaults.

Then click **OK**.

Create Form

Configure the components that you want to display in your form. Note that you can remove or edit the resulting components after you click OK. You can also add more components directly to the layout later.

☒ Read-Only Form
☐ Fields Generated Dynamically at Runtime

Include Controls for
☒ Row Navigation
☐ Submit

Fields:

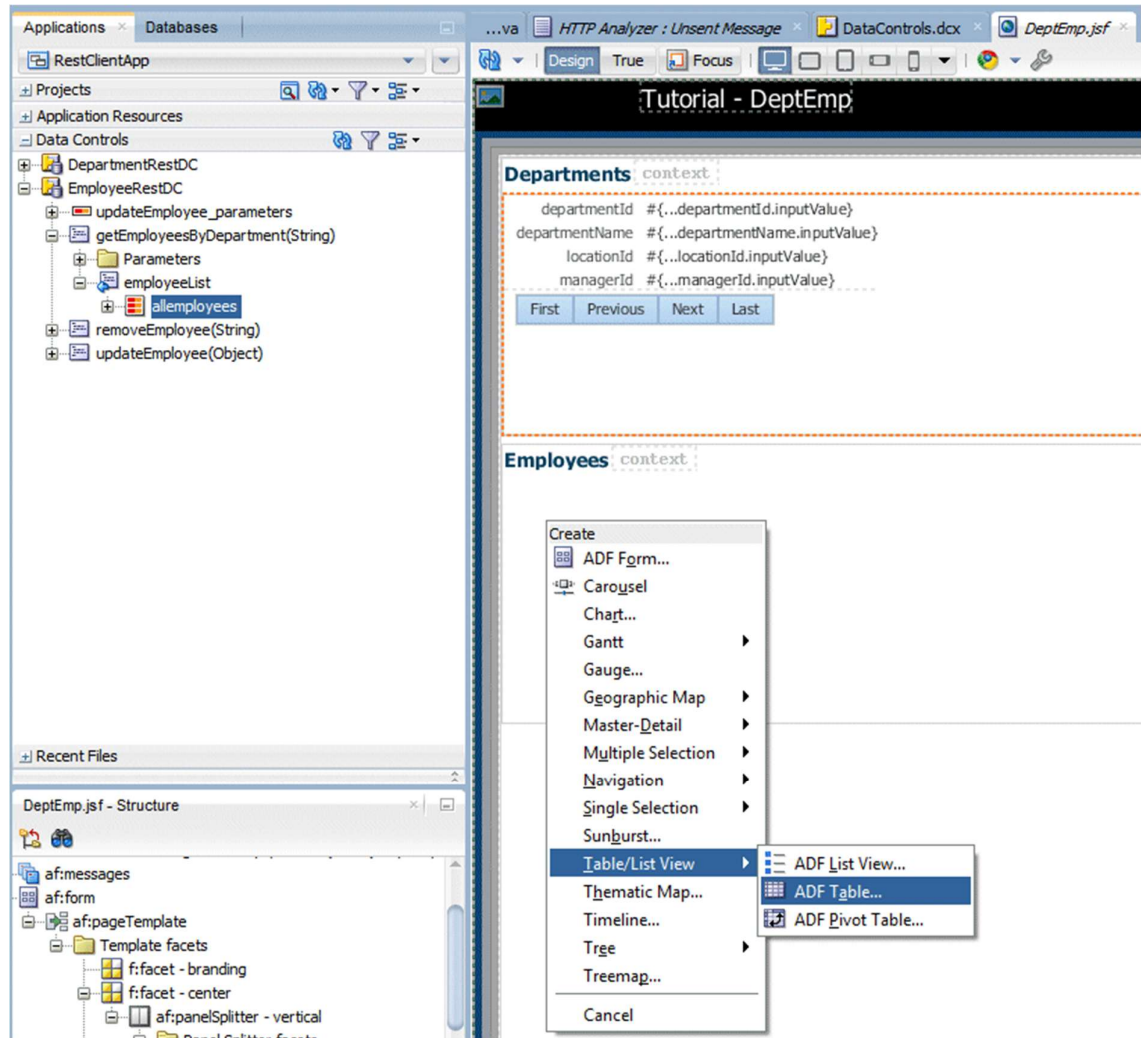
Component To Use	Value Binding	Display Label
ADF Input Text w/ Label	departmentId	<default>
ADF Input Text w/ Label	departmentName	<default>
ADF Input Text w/ Label	locationId	<default>
ADF Input Text w/ Label	managerId	<default>

Help OK Cancel

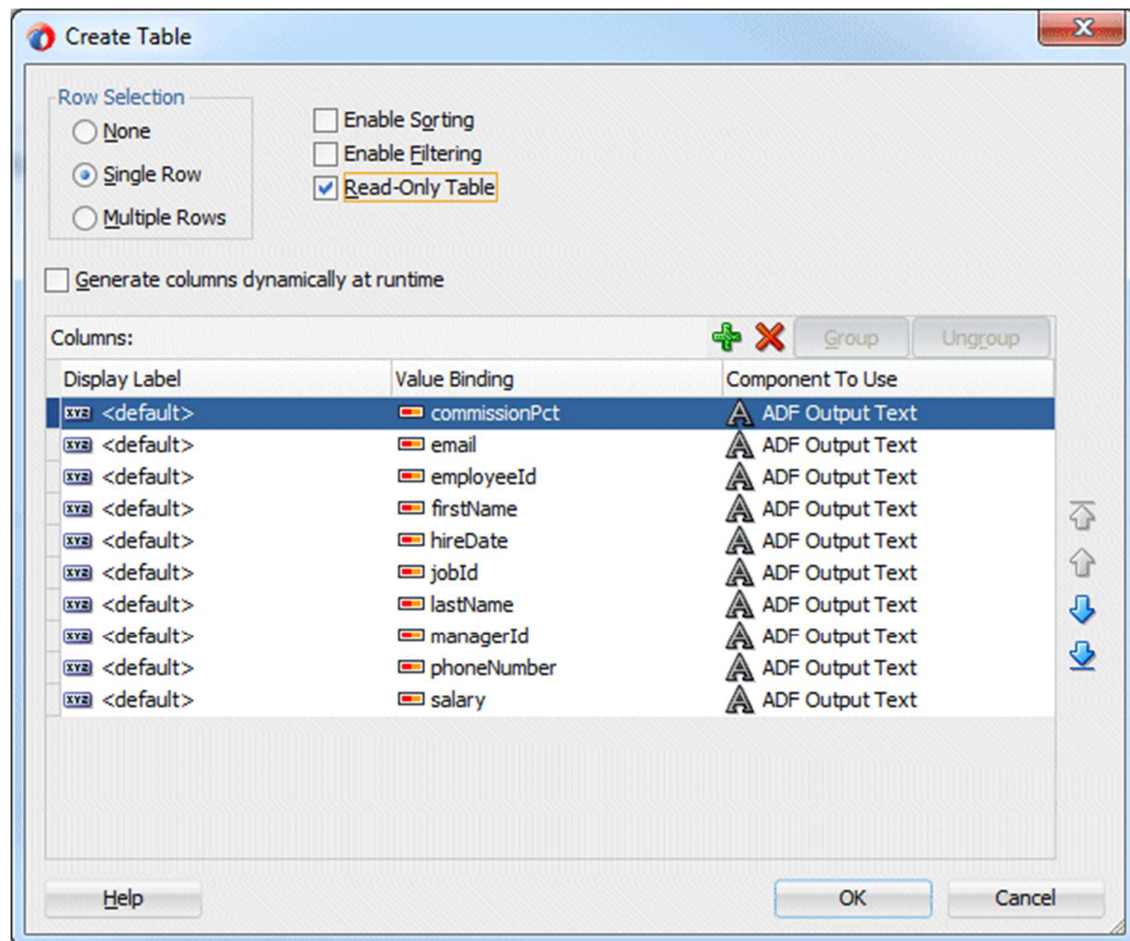
5. In the **Data Controls** panel, expand the **EmployeeRestDc > getEmployeesByDepartment > employeeList** node.

Drag the **allemployees** entry from the Data Controls panel, and drop it in the **Employees** area in the visual page editor.

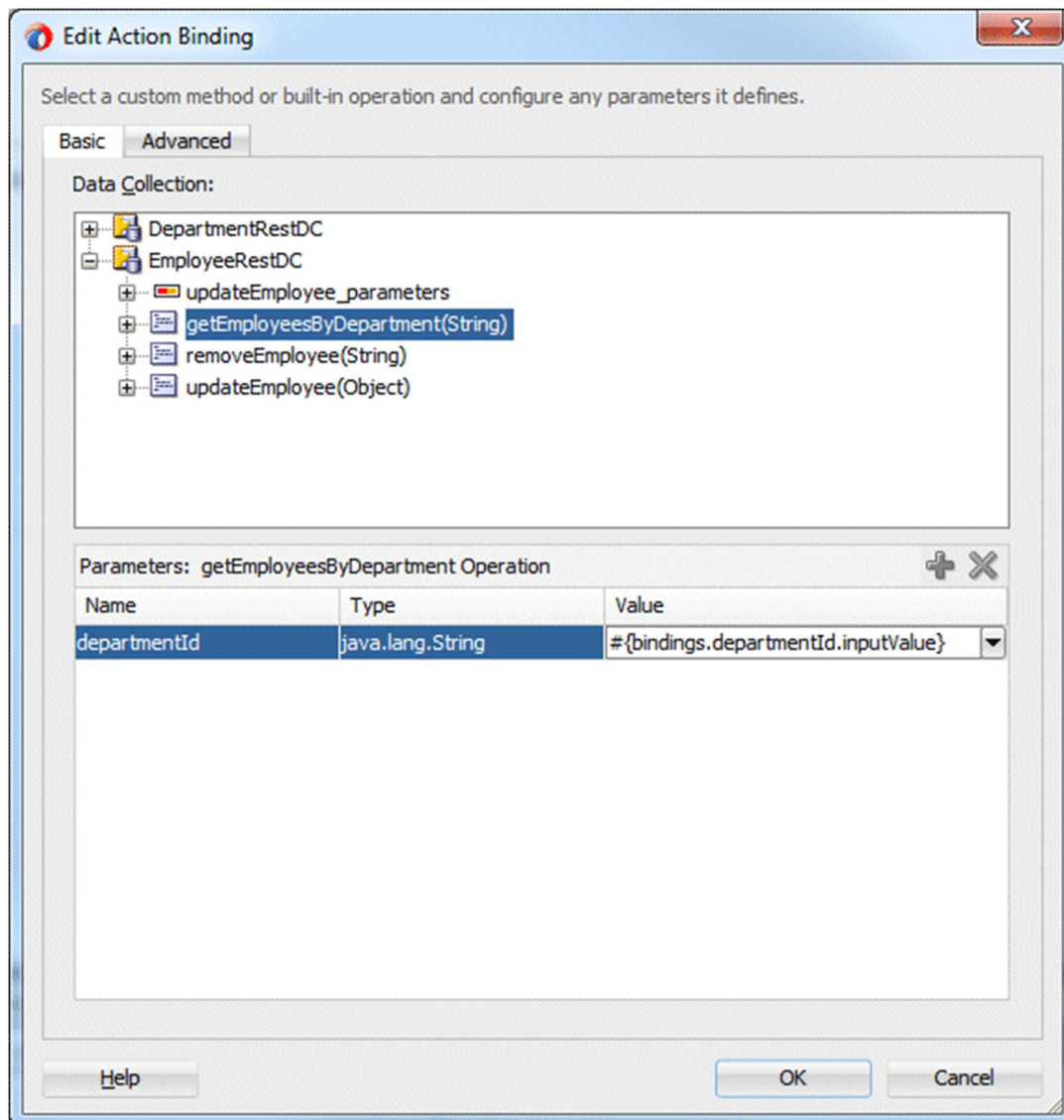
Select **Table/List View > ADF Table** from the Create menu..



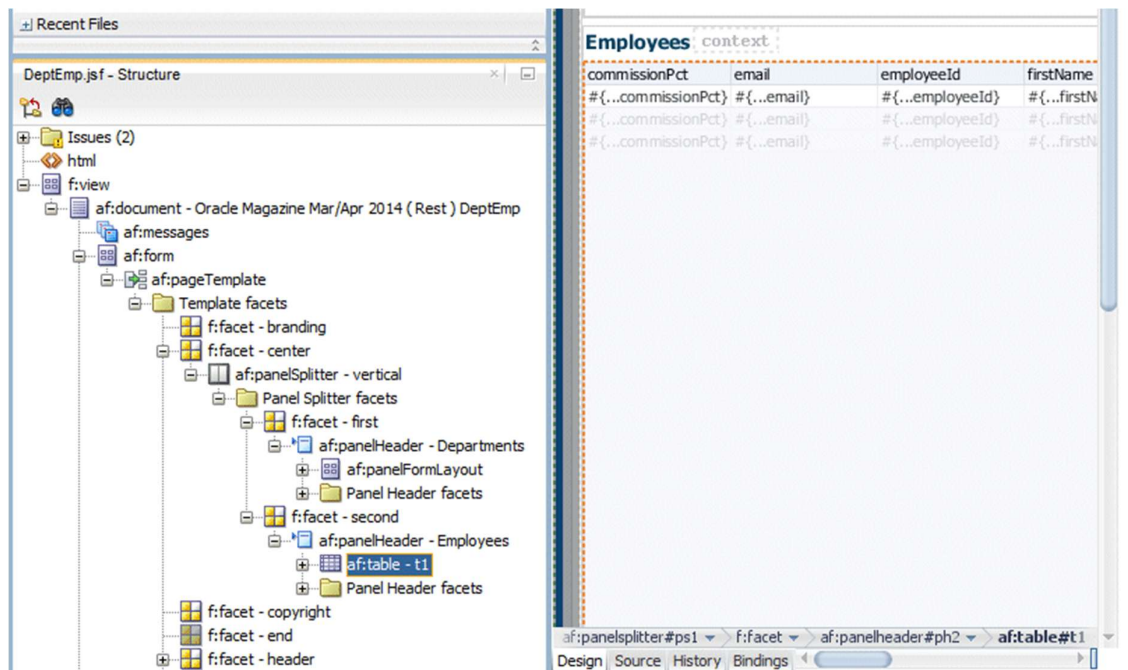
6. In the **Create Table** dialog box, select **Single Row** and **Read-Only Table**. Optionally, reorder the employee attributes so they're in a more logical order if you like. Click **OK**.



7. In the **Edit Action Binding** dialog box, enter `#{bindings.departmentId.inputValue}` as the **departmentId** value and click **OK**.

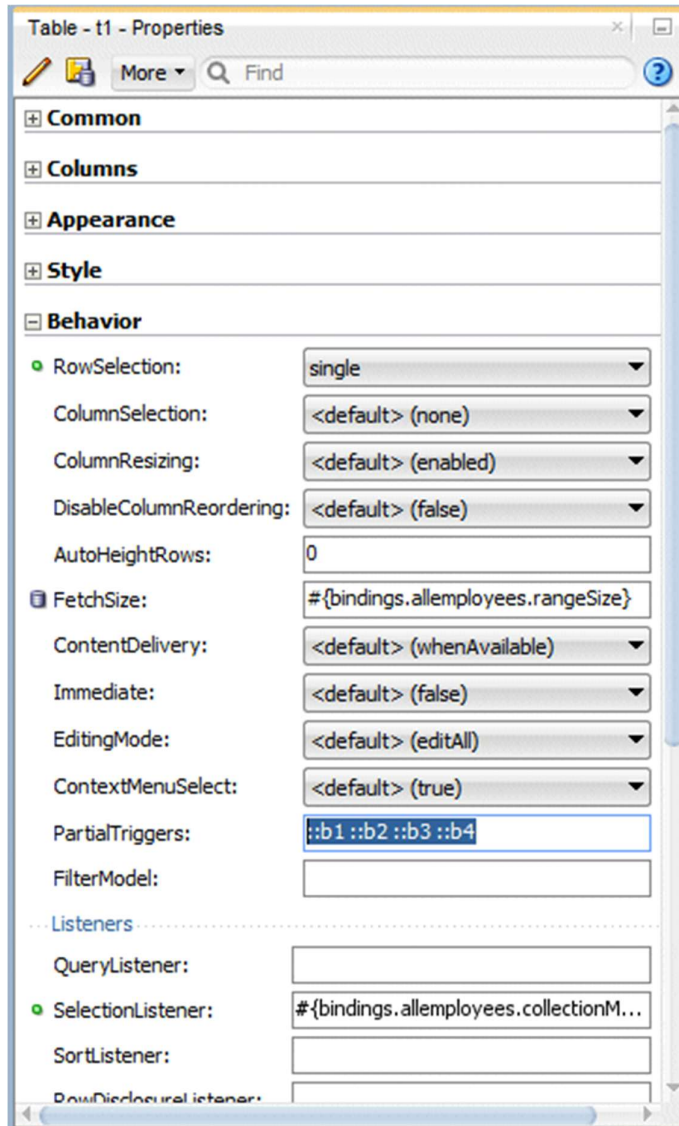


8. In the **Structure** window, click the **table** from the Employees area. The bread crumb at the bottom of the visual editor should be the **af:table#t1** crumb, highlighted (boldfaced), as the last element. If not, click the bread crumb to select the table.



9. Open the **Properties** window (**Window > Properties**), navigate to the **PartialTriggers** property under the Behavior category, and enter **::b1 ::b2 ::b3 ::b4**.

Note: ::b1 through ::b4 are the IDs of the navigation buttons that should trigger a table refresh. Whenever the user navigates the list of departments, the employee list will refresh to display only the employees of the selected department. .



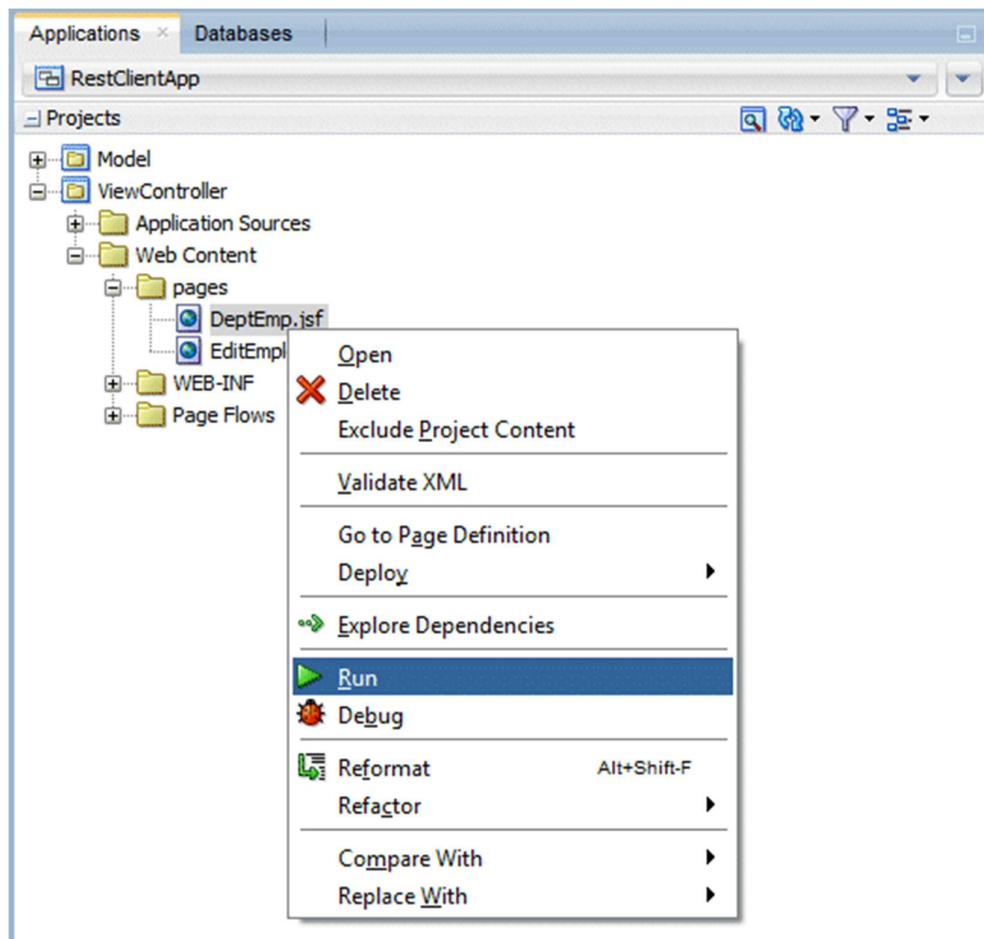
10. Save all your work.

In this step, you created a master-detail page that uses the department REST service, the department REST data control, and the employee REST data control you created in previous steps.

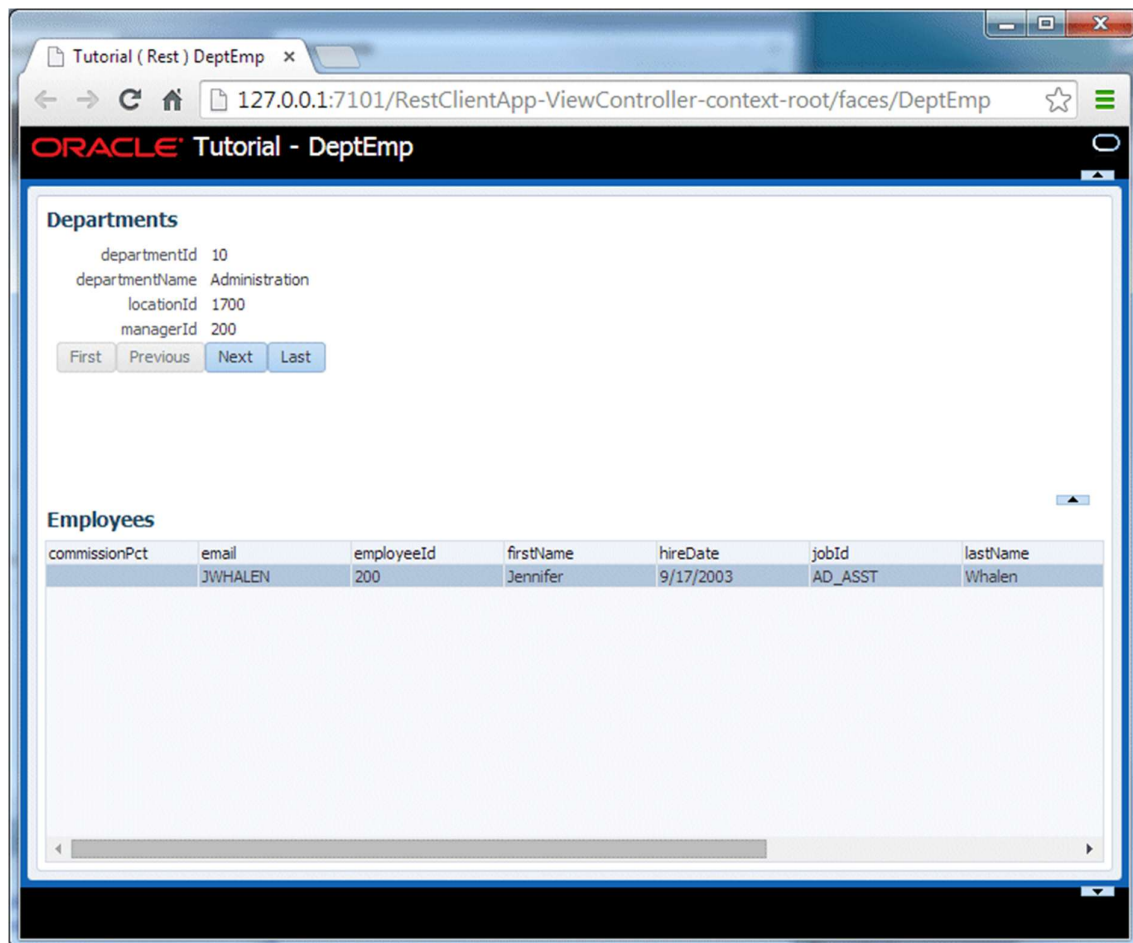
Step 4: Run and Test the Application

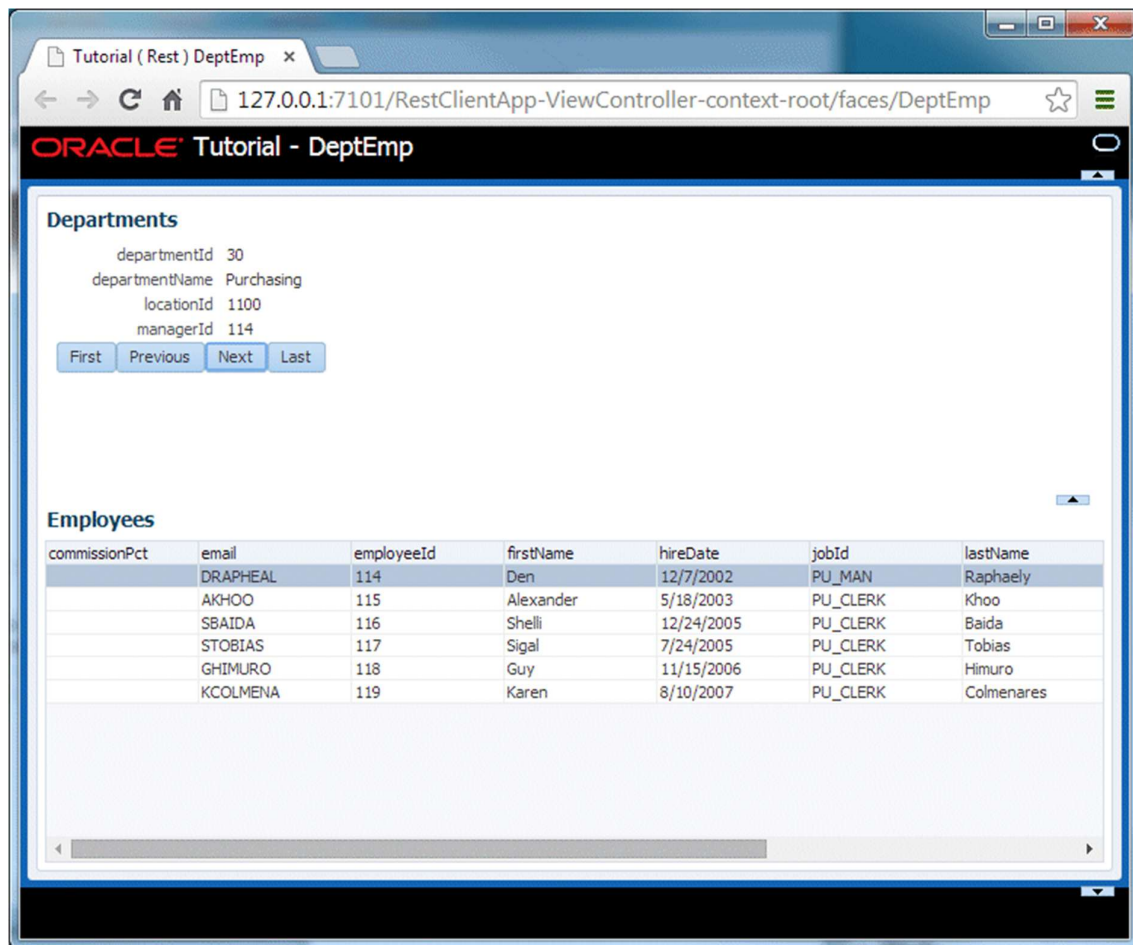
In this step, you run and test the REST service functionality.

1. In the Application panel, expand the **ViewController > Web Content > pages**, right-click the **DepEmp.jsf** file node and select **Run**.



2. Use the navigation buttons in the Departments section to change the display and watch the employee list refresh.





3. **Close** the browser window and return to JDeveloper. The tutorial is complete.

Part 3: Running the Complete Application

In this section you will open and run the completed application. This application uses all the REST services you created and consumed as data controls.

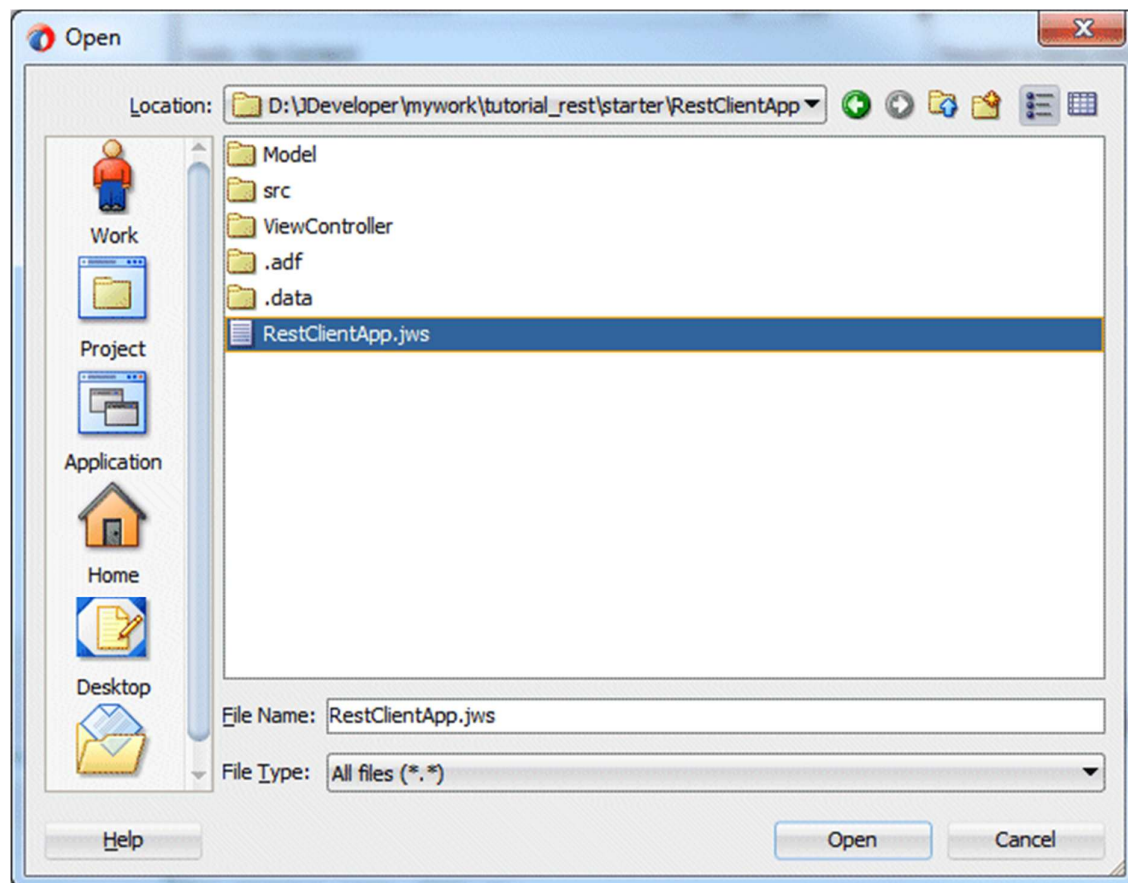
When the DeptEmp page runs you'll see the same Department master with Employees as the detail. A button named Edit Selected Employee will navigate to an EditEmployees page where you can modify an employee and save your changes.

Step 1: Open and Run the Completed Client Application

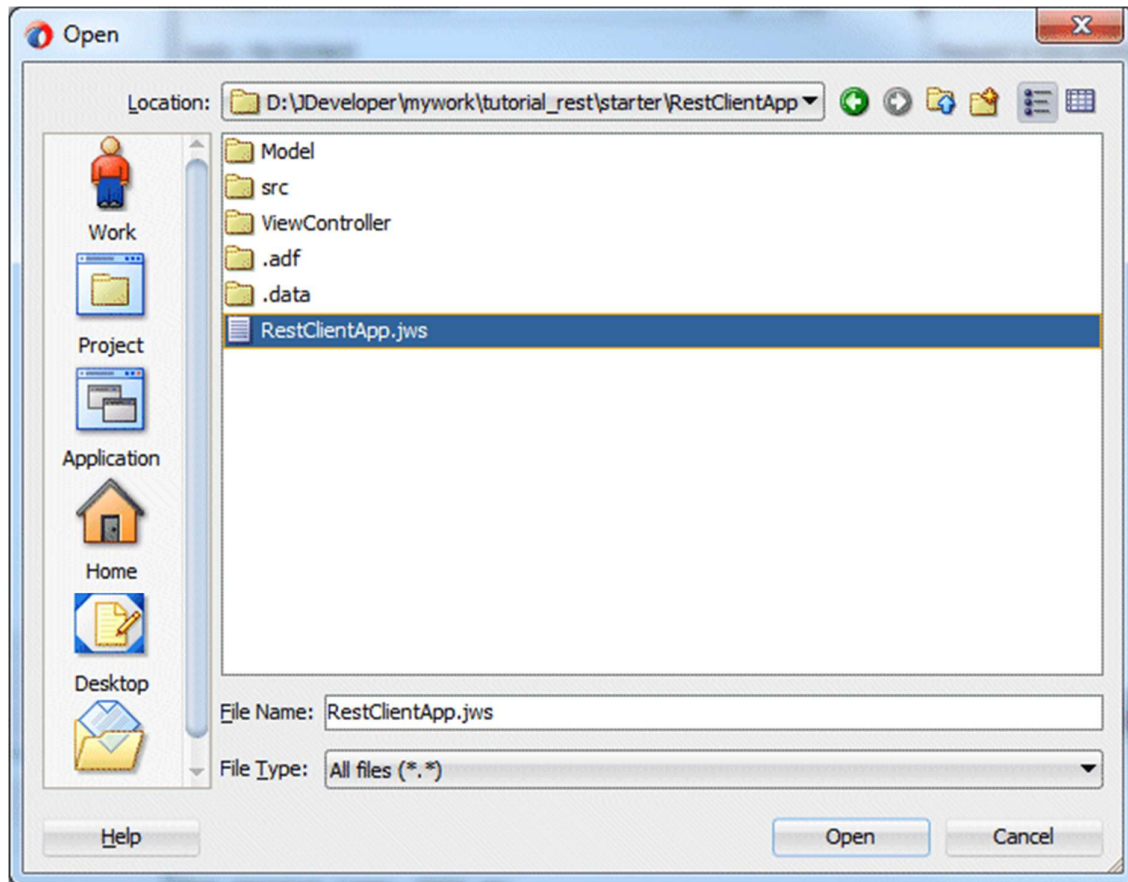
In the next few steps, you'll open the completed client application. The client application will run with your EjbRestService (Model/RestService) application. You MUST have the EjbRestServices (<http://localhost:7101/hr-restservice-context-root/resources/tut/department>) running for the completed client application to retrieve records.

If they are not started, start both the DepartmentsService and EmployeesService.

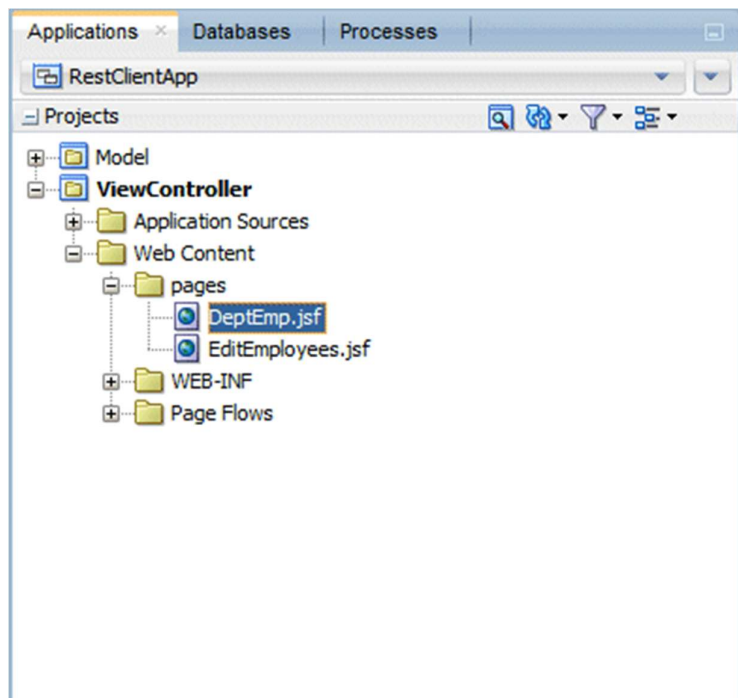
1. Download the [completed application zip](#) file and extract to a location that works best for you.



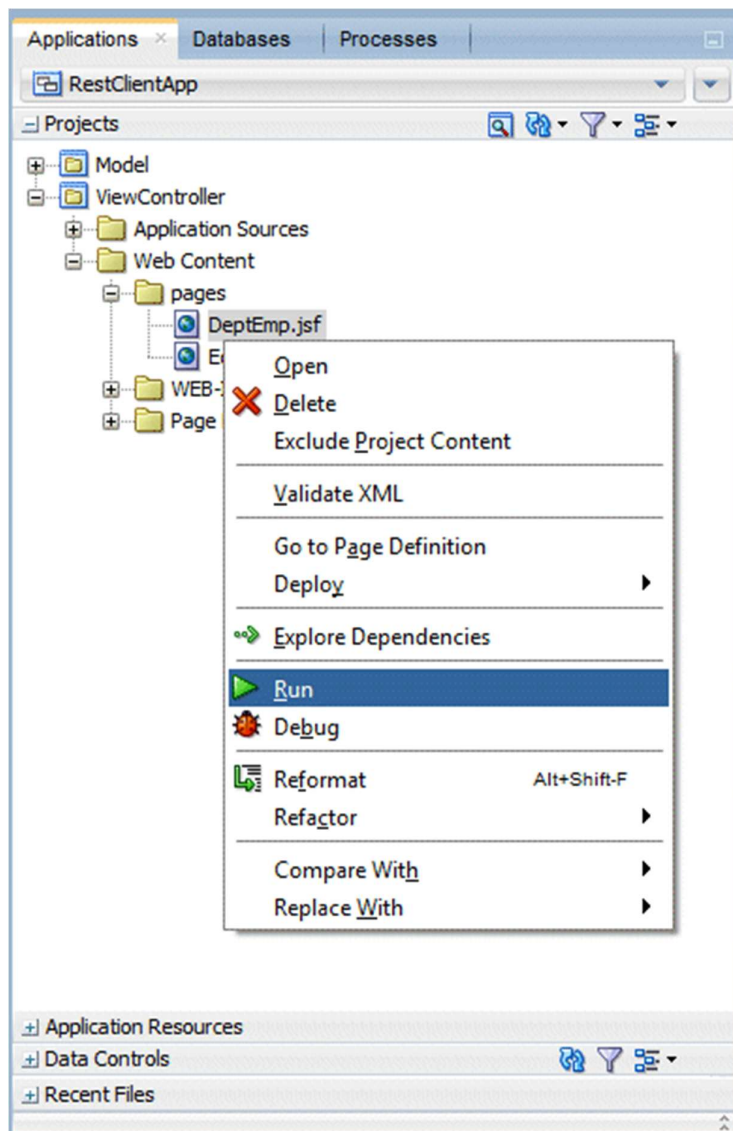
2. In JDeveloper, select **File > Open** and navigate to the directory containing the unpacked sample application zip file content. Open the **completed\RestClientApp** folder and select the **RestClientApp.jws** file.



3. In the **Application** window, expand the **ViewController > Web Content > pages** and select **DeptEmp.jsf**.



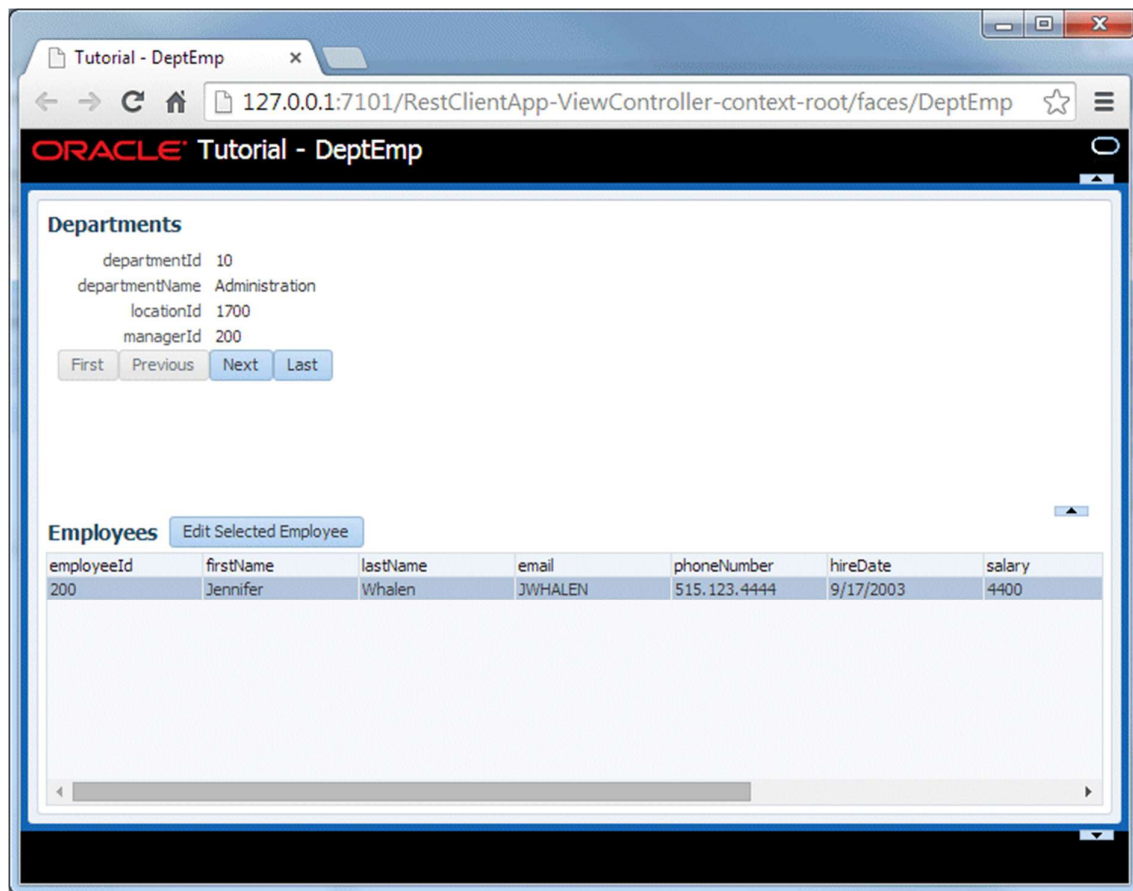
4. Right click the **DeptEmp.jsf** and from the context menu, select **Run**.



Step 2: Test the Application

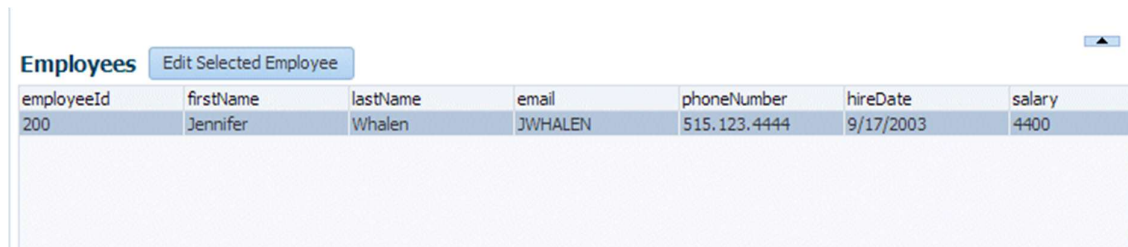
In the next few steps, you examine the completed application and test the REST service functionality.

1. Once the application opens in the browser it should look very similar to the one you saw upon completion of Part 2: Developing the Client Application, with one difference.



2. You still have the scrolling between department record, but you should also see a button next to the Employees table label.

With an employee highlighted in the table, click the **Edit Selected Employee** button.



3. Click the the button and you navigate to a second page which displays the employee details.

Oracle Magazine Mar/Apr x

127.0.0.1:7101/RestClientApp-ViewController-context-root/faces/EditEmployee

ORACLE Oracle Magazine Mar/Apr 2014 (Rest) Edit Emp

employeeId	200
firstName	Jennifer
lastName	Whalen
hireDate	9/17/2003
email	JWHALEN
phoneNumber	515.123.4444
jobId	AD_ASST
salary	4400
commissionPct	
managerId	101

Submit

4. Modify any of the field values, except the employeeId.

In the image below, the lastName has been modified.

Oracle Magazine Mar/Apr x

127.0.0.1:7101/RestClientApp-ViewController-context-root/faces/EditEmployee

ORACLE Oracle Magazine Mar/Apr 2014 (Rest) Edit Emp

employeeId 200

firstName Jennifer

lastName Whalen-Smith

hireDate 9/17/2003

email JWHALEN

phoneNumber 515.123.4444

jobId AD_ASST

salary 4400

commissionPct

managerId 101

Submit

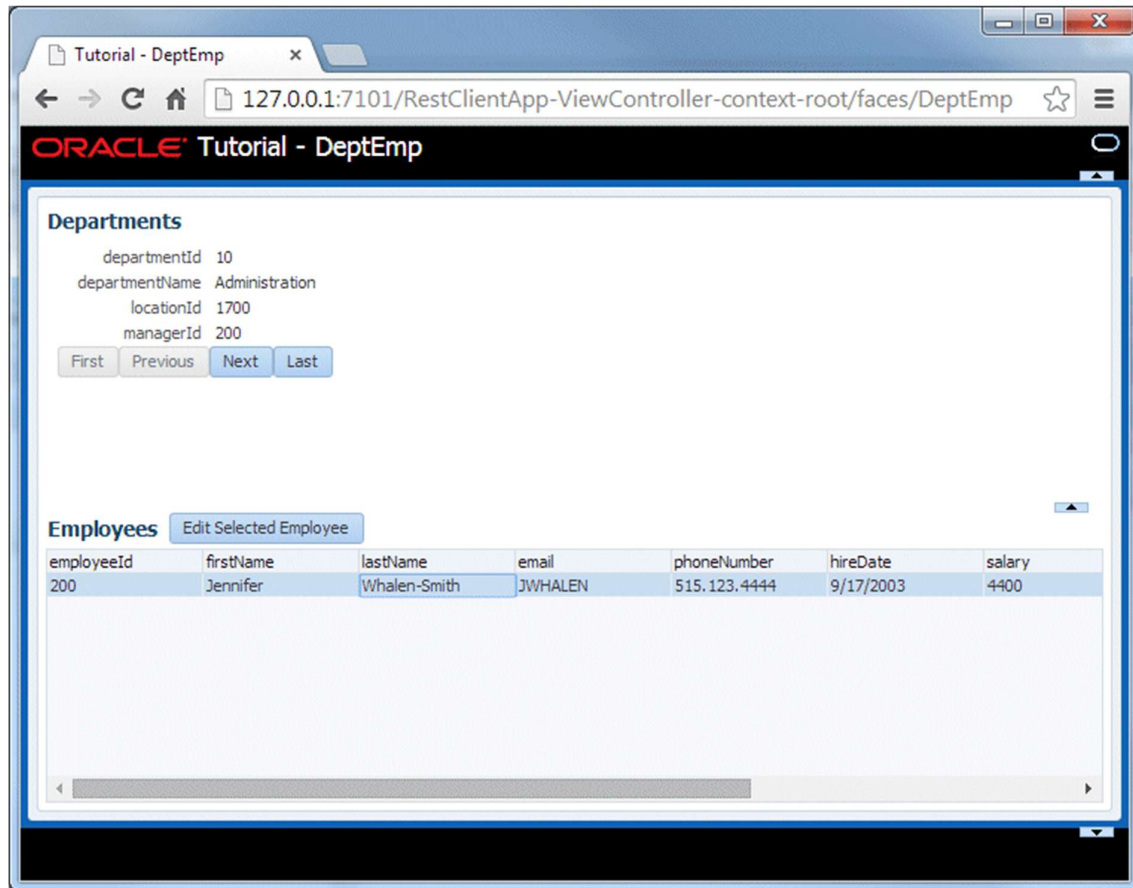
5. Then click the Submit button.

commissionPct

managerId

Submit

6. The application will return to the DeptEmp page and the lastName field reflects the change made on the previous page.



7. Close the browser. This part of the tutorial is complete